

CNNs

Gunhee Kim



서울대학교
SEOUL NATIONAL UNIVERSITY

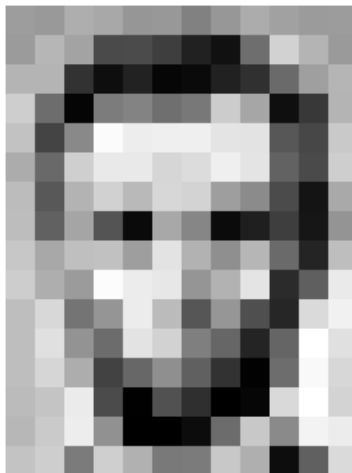
Outline

- Filters and Convolution
- Convolutional Neural network
- Case Studies

Filters in Images

- It is a basic tool for image processing
 - Transform images, or extract meaningful information from digital images via algorithms using computers
- Digital image as a function
 - $f(x, y)$ gives the intensity $[0, 255]$ at position (x, y)

$$f: \mathbb{R}^2 \rightarrow \mathbb{R}$$



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	83	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	228	227	87	71	201	
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	133	143	95	60	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	83	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	228	227	87	71	201	
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	133	143	95	60	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

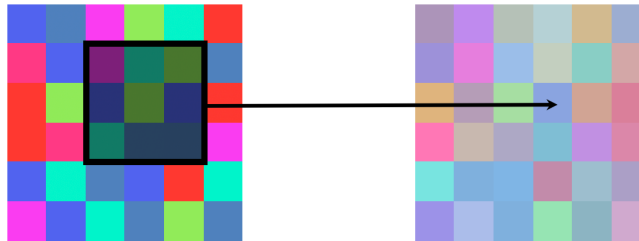
cf. A color image
is a function

$$f: \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$f(x, y) = \begin{bmatrix} r(x, y) \\ g(x, y) \\ b(x, y) \end{bmatrix}$$

Filters in Images

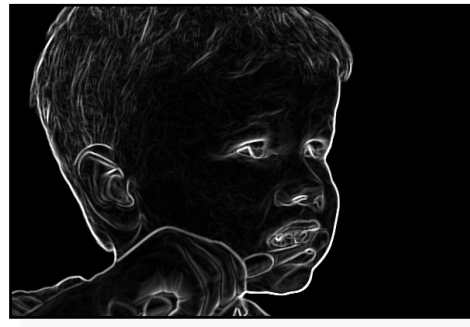
- Filters change the pixel value of images
 - Apply a function to digital image
 - Modify image properties and/or to extract valuable information
 - ex. Noise reduction, blurring, sharpening, edge
- Neighborhood operation



Convolution!



Image



Edge detection



Blur

Convolution

- Multiply each kernel (filter) value by the corresponding pixel value

Input Kernel Output

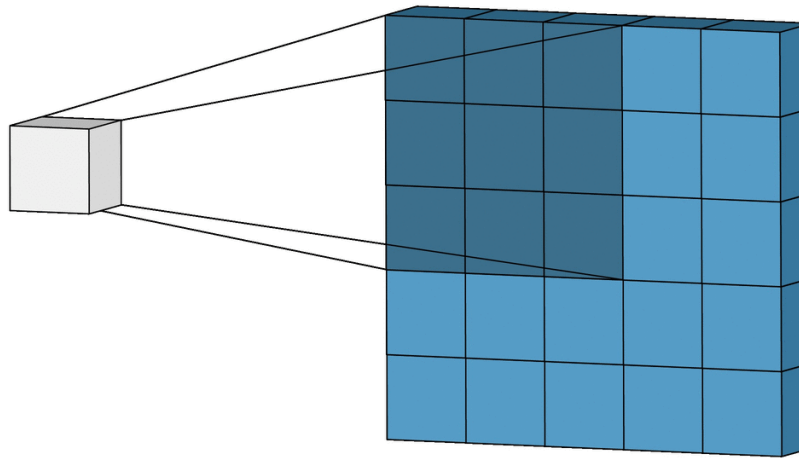
0	1	2
3	4	5
6	7	8

 $*$

0	1
2	3

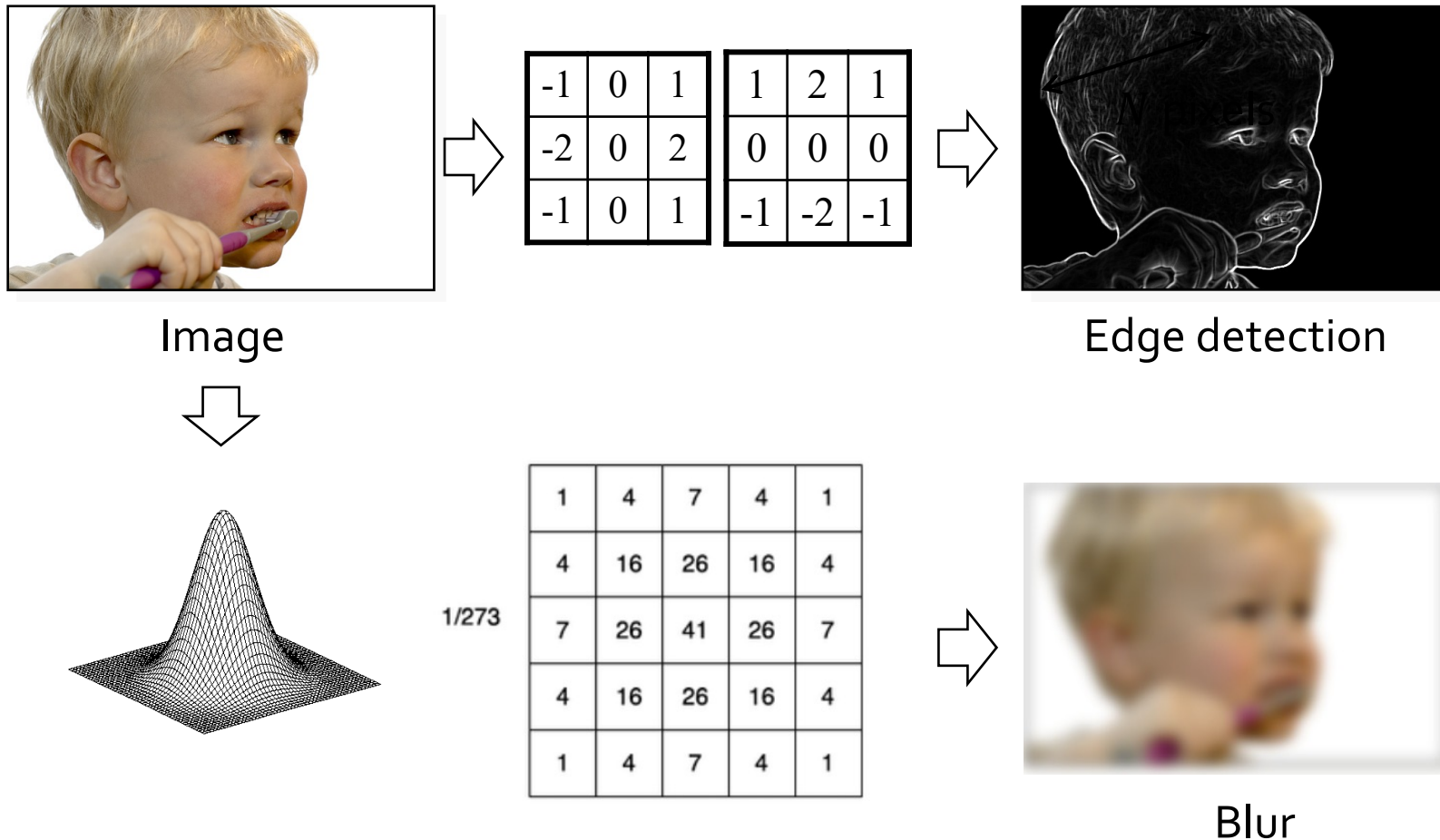
 $=$

19	25
37	43

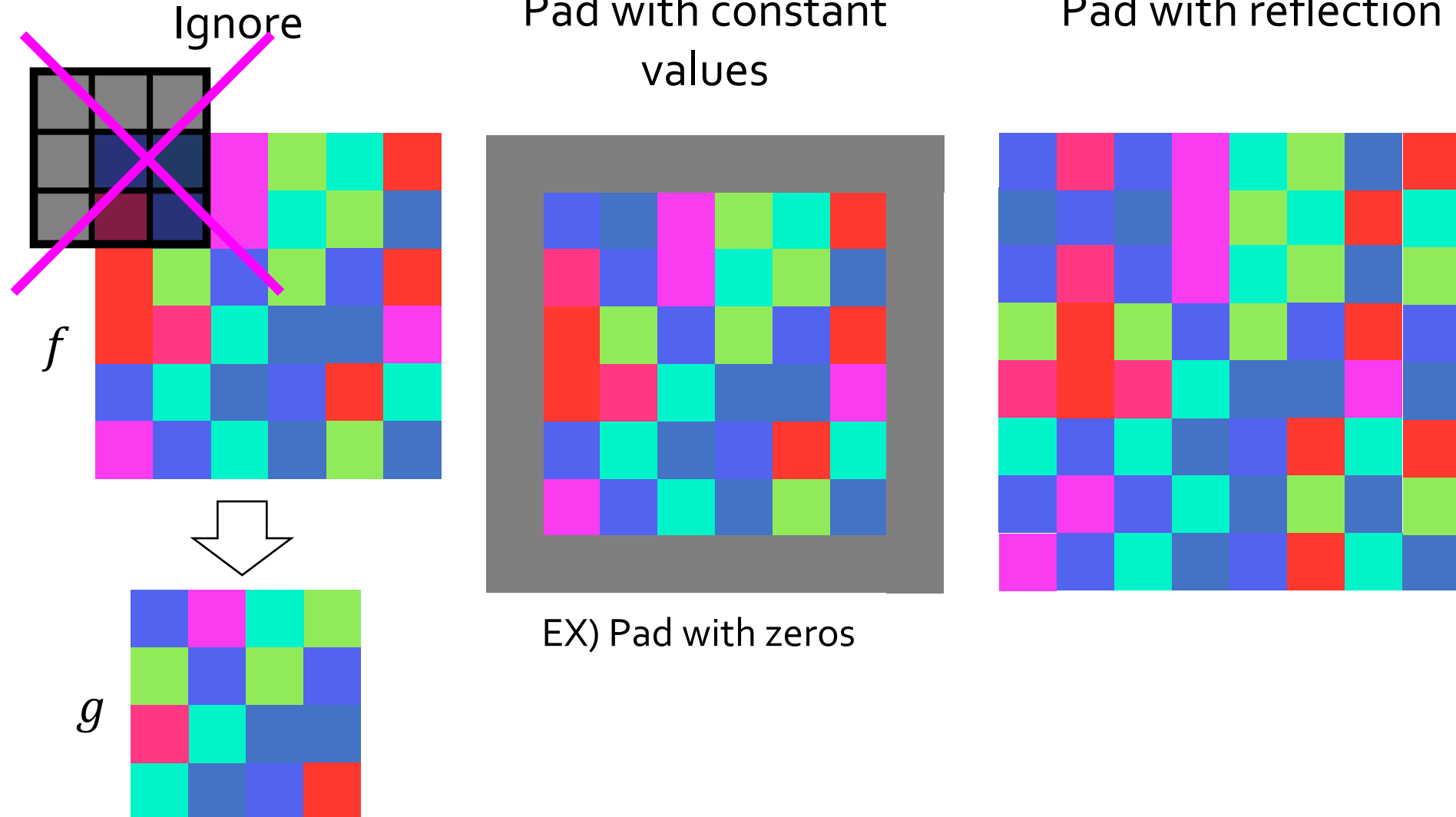


Convolution

- Image processing can be done by convolution with filters (kernels)

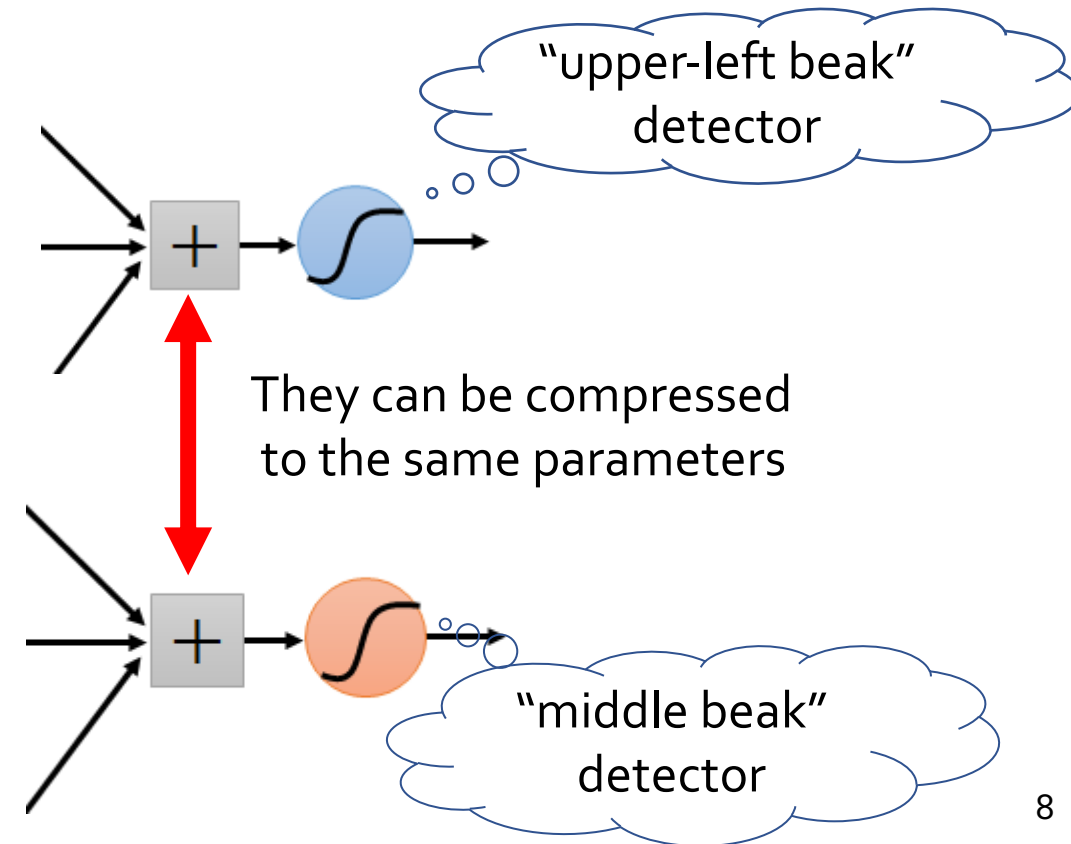
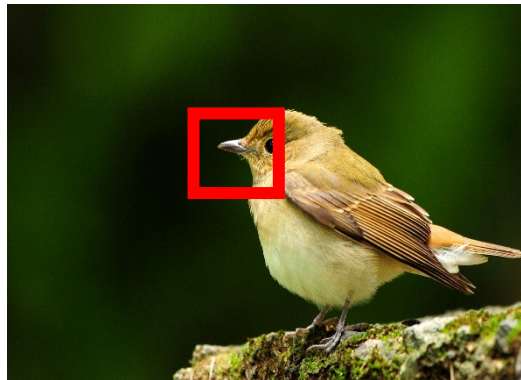


Convolution – Border Problem



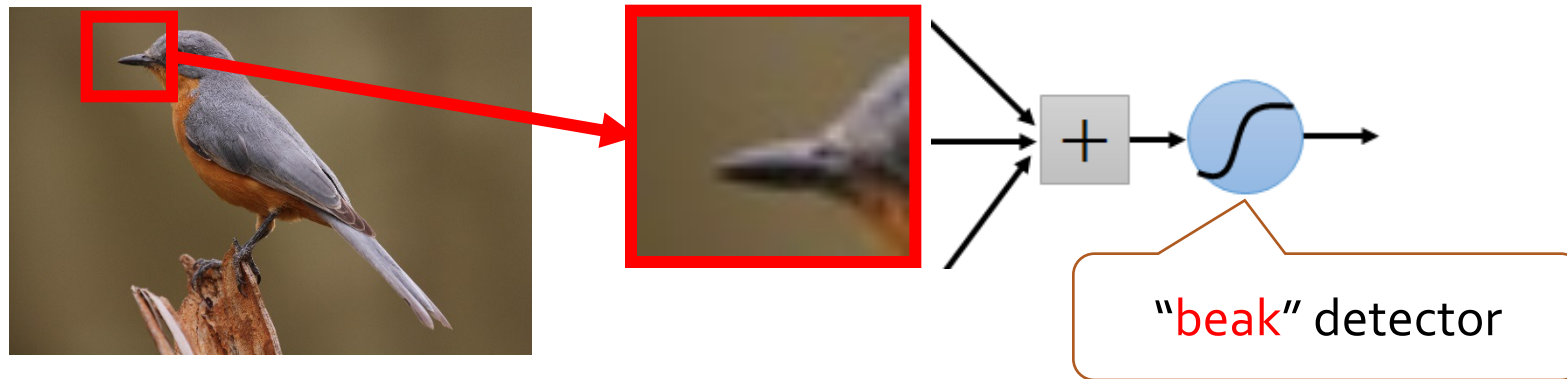
Why Do CNNs Need Filters?

- Some patterns for recognition are much smaller than the whole image
 - Also want to represent a small region with fewer parameters
 - Beak detector is a filter in some upper layers of CNN!
- What about training a lot of such **small** detectors and each detector must move around?



Why Do CNNs Need Filters?

- However, we do not know which patterns are important to detect a bird beforehand
- Let CNNs learn a set of filters!
 - More correctly, a hierarchy of sets of filters



Outline

- Filters and Convolution
- Convolutional Neural network
- Case Studies

Convolutional Neural Network (ConvNet)

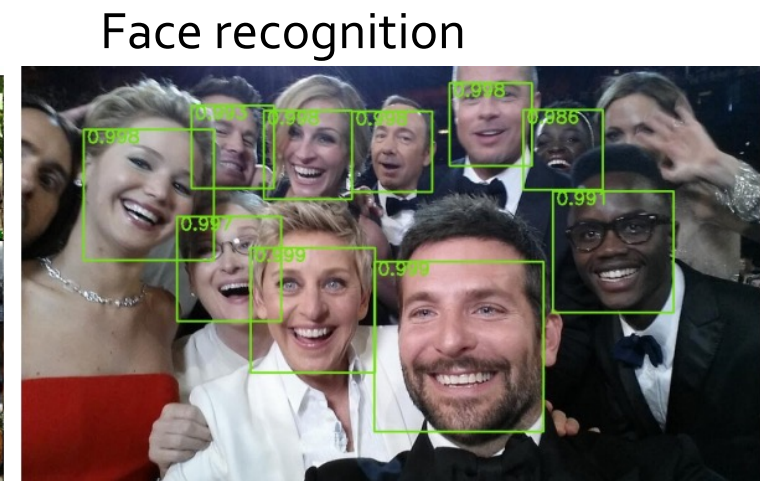
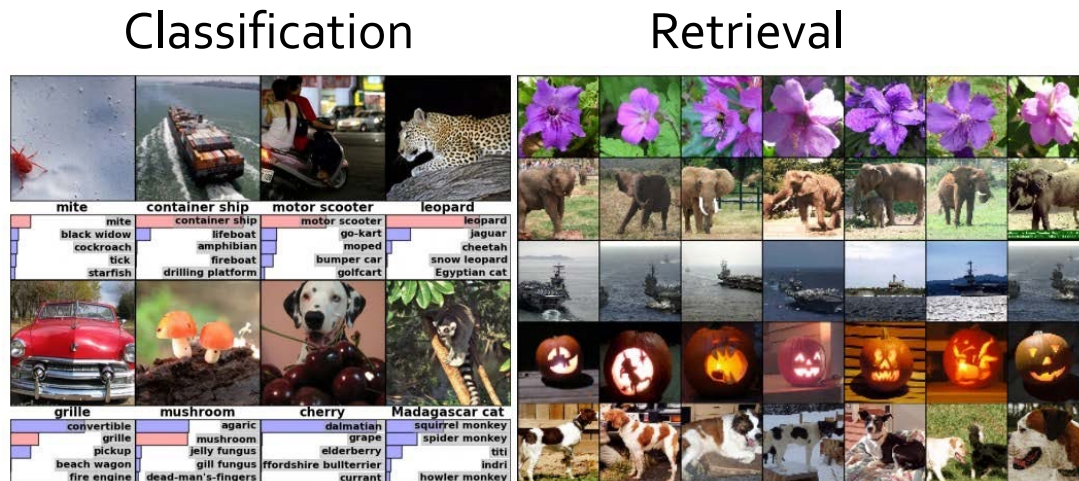
- Some background knowledge
 - Convolution of the image with filters
 - Backpropagation + gradient based optimization
 - A ConvNet
 - Consists of many convolutional + subsampling (i.e., pooling) + fully-connected layers
 - In most cases, input is a 2D images and output is class labels
 - ConvNets are everywhere in computer vision!
- Classification

mite container ship motor scooter leopard

mite black widow cockroach tick starfish
container ship lifeboat amphibian fireboat drilling platform
motor scooter go-kart moped bumper car golfcart
leopard jaguar cheetah snow leopard Egyptian cat

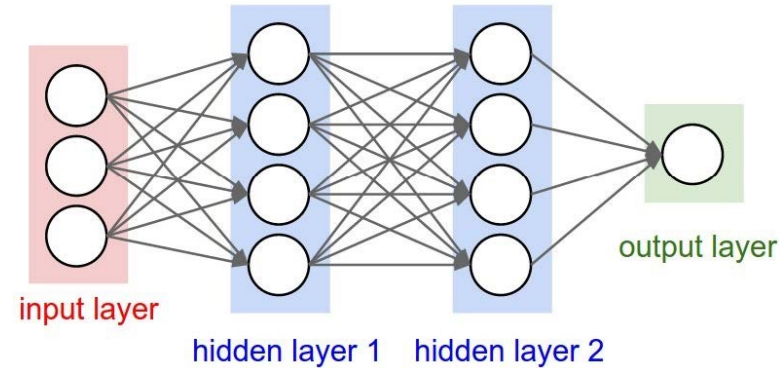
Retrieval

Face recognition

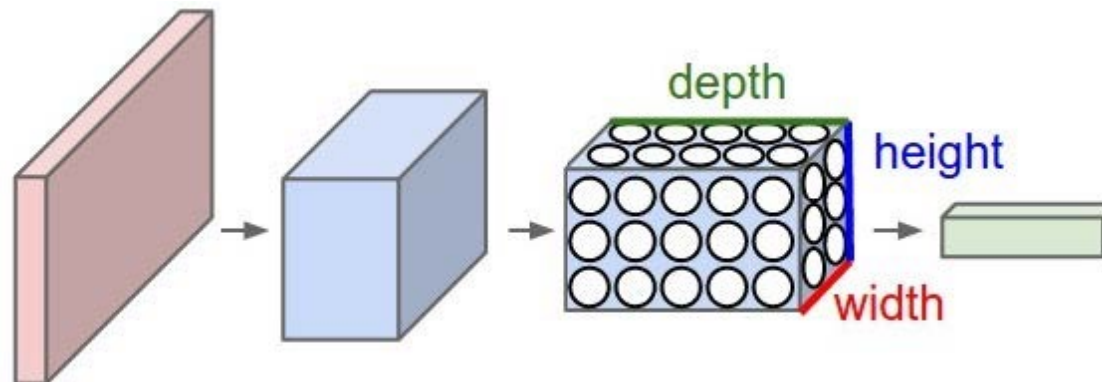


ConvNet Representation

- In previous multi-layer NN



- Each layer of ConvNet is represented by 3D volume of neurons (width, height, depth)

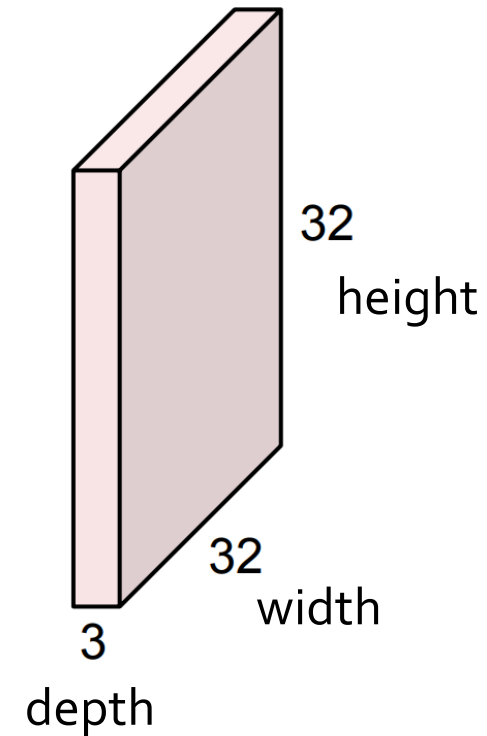
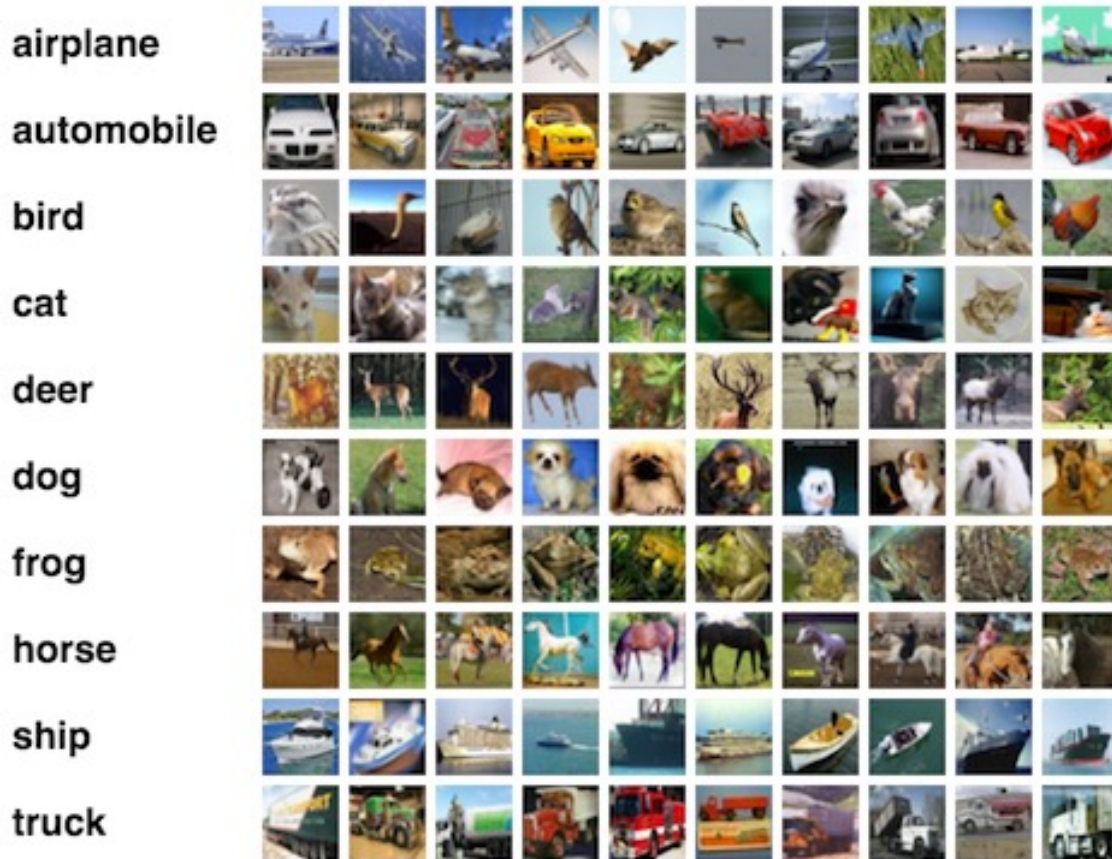


ConvNet Representation

- 1. INPUT layer: raw pixel values of the image
- 2. COV (Convolutional) layer: output of neurons that are connected to local regions in the input
- 3. Activation layer (RELU): element-wise activation function
- 4. POOL layer: downsampling operation
- 5. FC (Fully-connected) layer: computing a class score
- Parameters
 - CONV/FC have parameters (weights and bias of neurons)
 - RELU/POOL have NO parameters

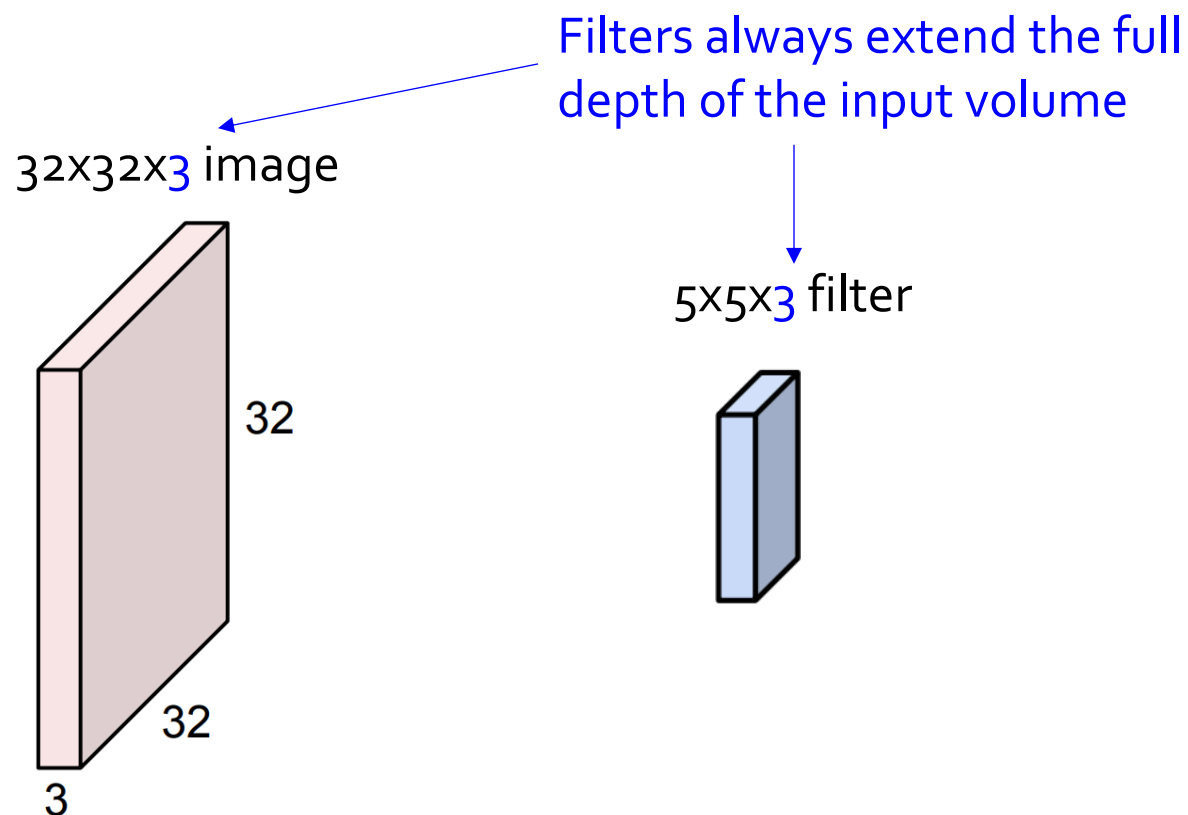
Input Layer

- Ex. CIFAR-10: Images of 10 classes with $32 \times 32 \times 3$ (RGB)
 - Input layer (Raw pixels of an image)



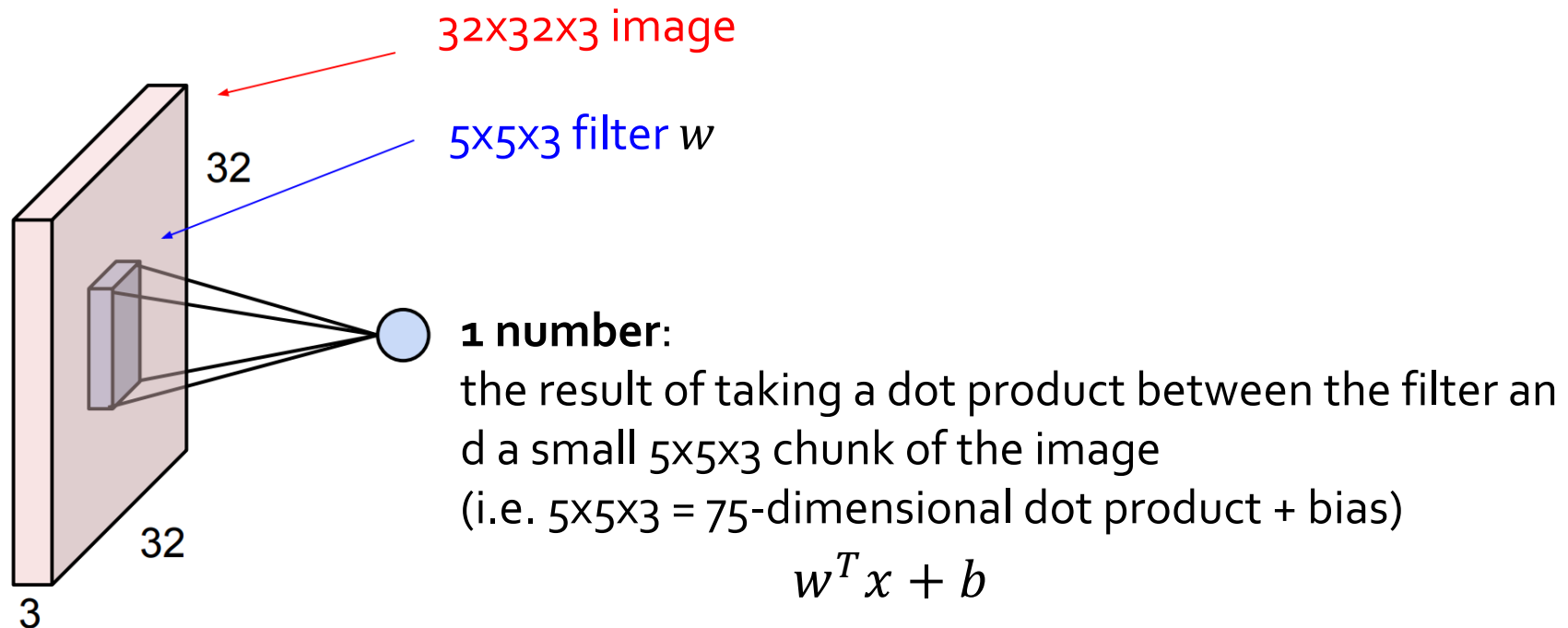
Convolution Layer

- Convolve the filter with the image
 - Slide over the image spatially, computing dot products



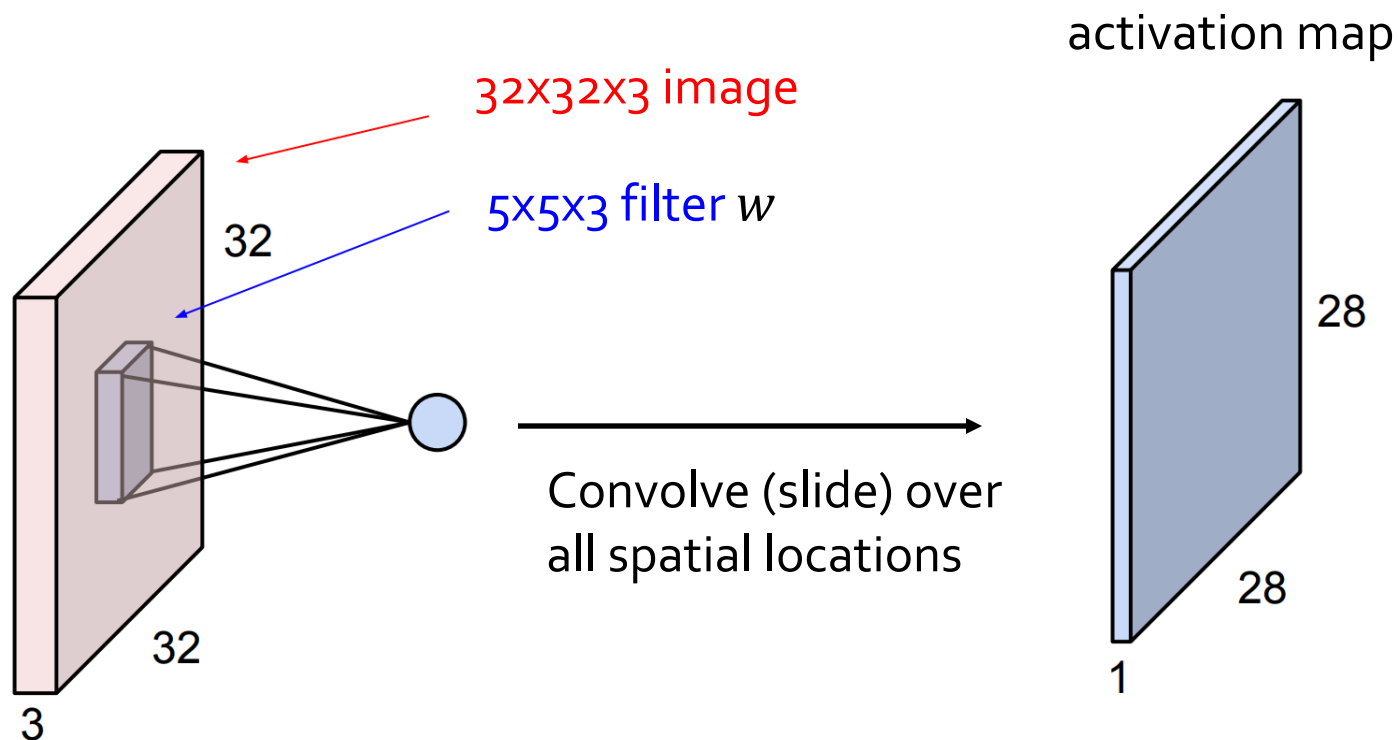
Convolution Layer

- Convolve the filter with the image
 - Slide over the image spatially, computing dot products



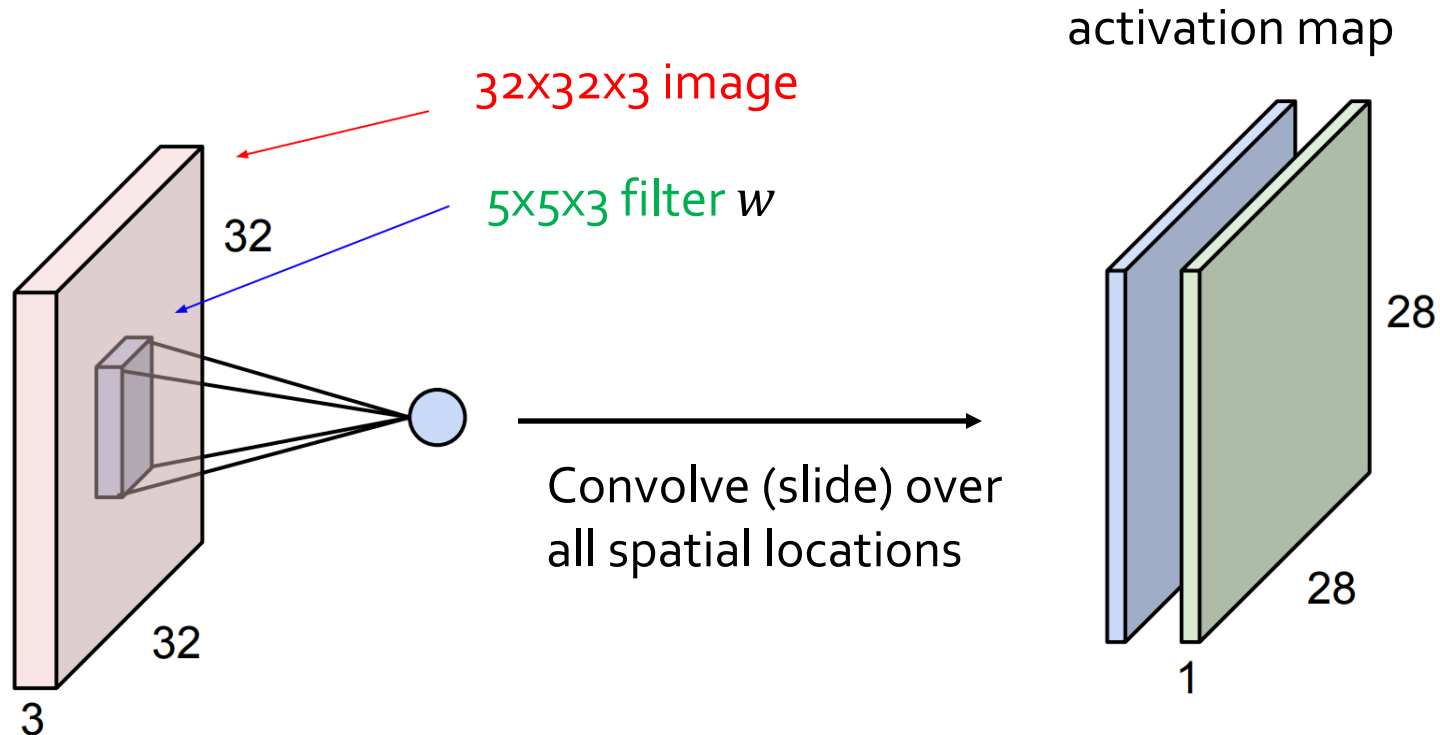
Convolution Layer

- Convolve the filter with the image
 - Slide over the image spatially, computing dot products



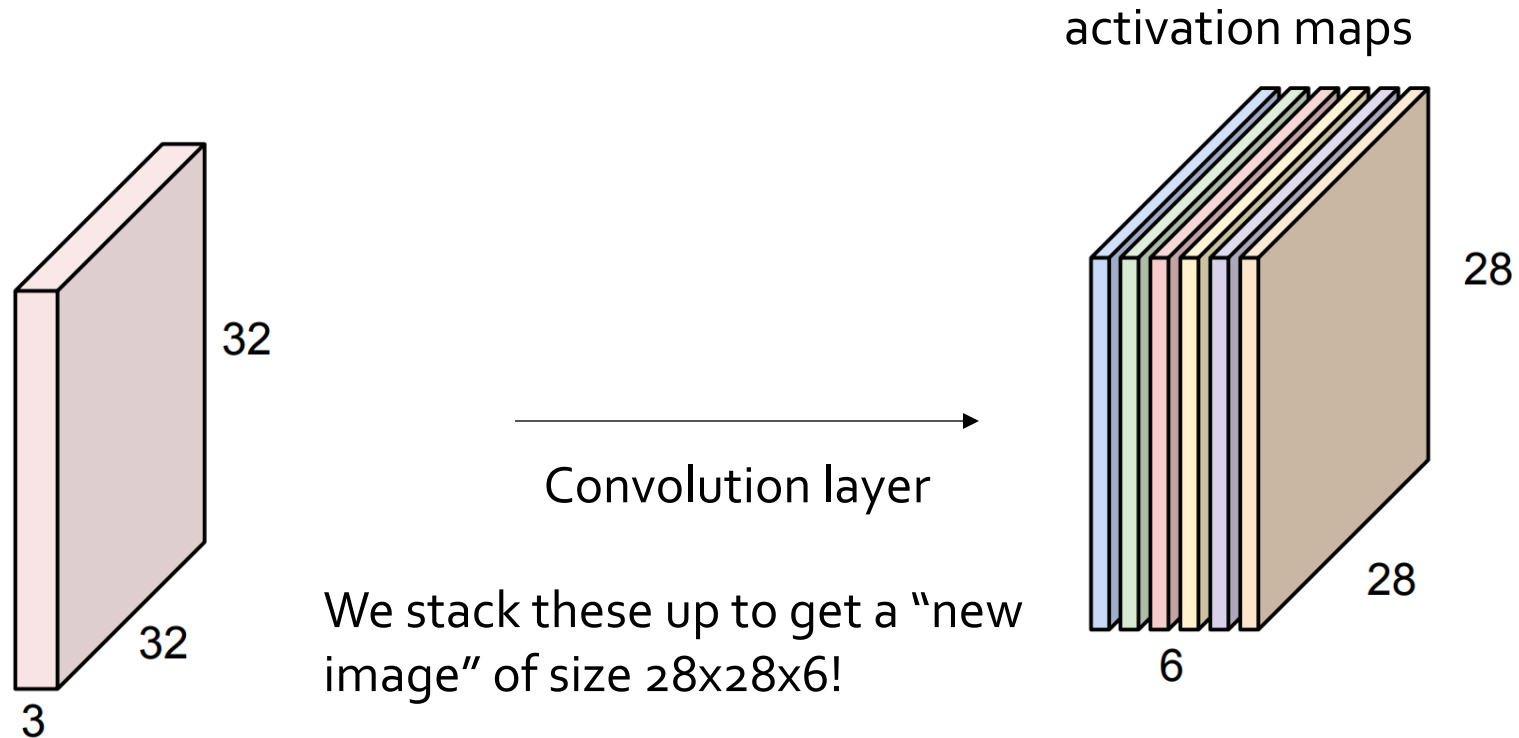
Convolution Layer

- Convolve the filter with the image
 - Slide over the image spatially, computing dot products
 - Consider a second, **green** filter



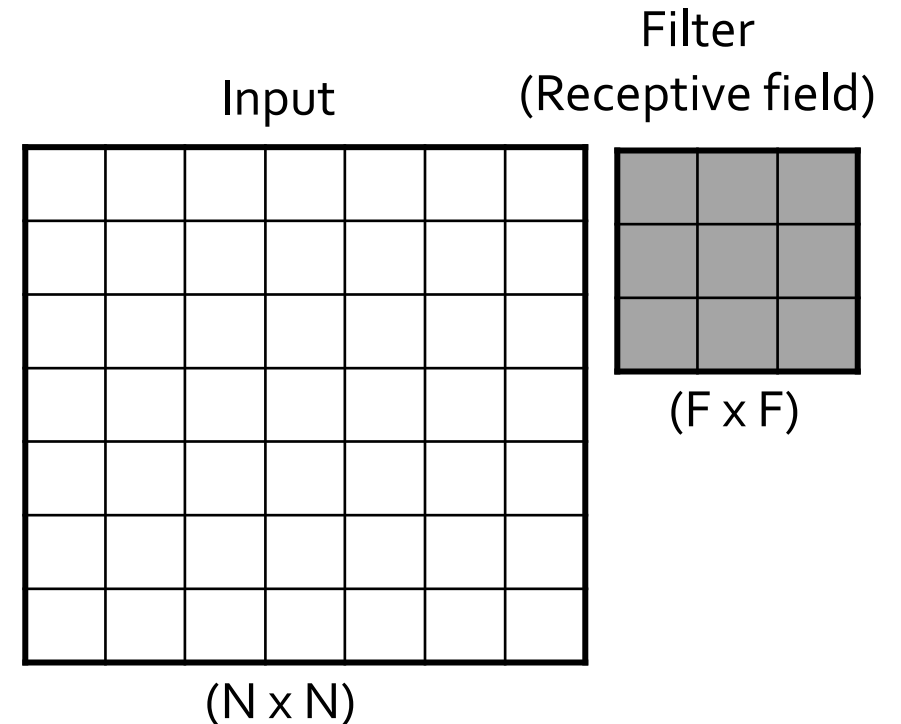
Convolution Layer

- If we had 6 5×5 filters, we'll get 6 separate activation maps
 - The 6 5×5 filters should be learned from data during training
 - a.k.a. kernels, receptive fields



Convolution Layer

- A closer look at spatial dimension
- Stride: distance between consecutive filter application
- An example
 - 7x7 input (spatially) and 3x3 filter
 - Output size: $(N - F)/S + 1$
 - e.g., Stride 1 $\rightarrow (7-3)/1+1=5$
Stride 2 $\rightarrow (7-3)/2+1=3$
Stride 3 $\rightarrow (7-3)/3+1=2.33?$



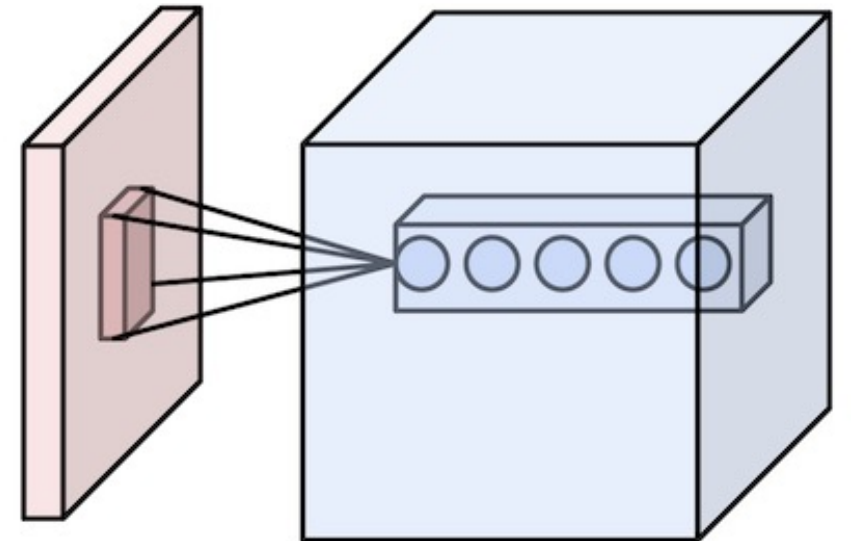
Convolution Layer

- In practice: common to zero pad the border
 - Pad the input with zeros spatially on the border
 - In general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F - 1)/2$ (will preserve size spatially)
 - Input 7×7 , filter 3×3 , stride 1, zero-pad 1.
What is the output?
 - $7 \times 7 \rightarrow$ Preserved size!
 - e.g., $F=3 \rightarrow$ zero-pad with 1
 $F=5 \rightarrow$ zero-pad with 2
 $F=7 \rightarrow$ zero-pad with 3
- No headaches when sizing architecture

0	0	0	0	0				
0								
0								
0								
0								

Example of Convolution Layer

- Question
 - Input volume: $32 \times 32 \times 3$
 - Filters: 5×5 , stride 1
 - Number of filters: 5
- Answer
 - Output size: $(32 - 5) / 1 + 1 = 28$
 - Output volume: $28 \times 28 \times 5$
 - How many weights are for each filter?
 $5 \times 5 \times 3 = 75$



Activation Layer

- An activation layer follows every CONV layer
 - The most common activation function is ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- ReLU's purpose is to introduce non-linearity in our ConvNet
- Point-wise operation with no parameter

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.0	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.0	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77

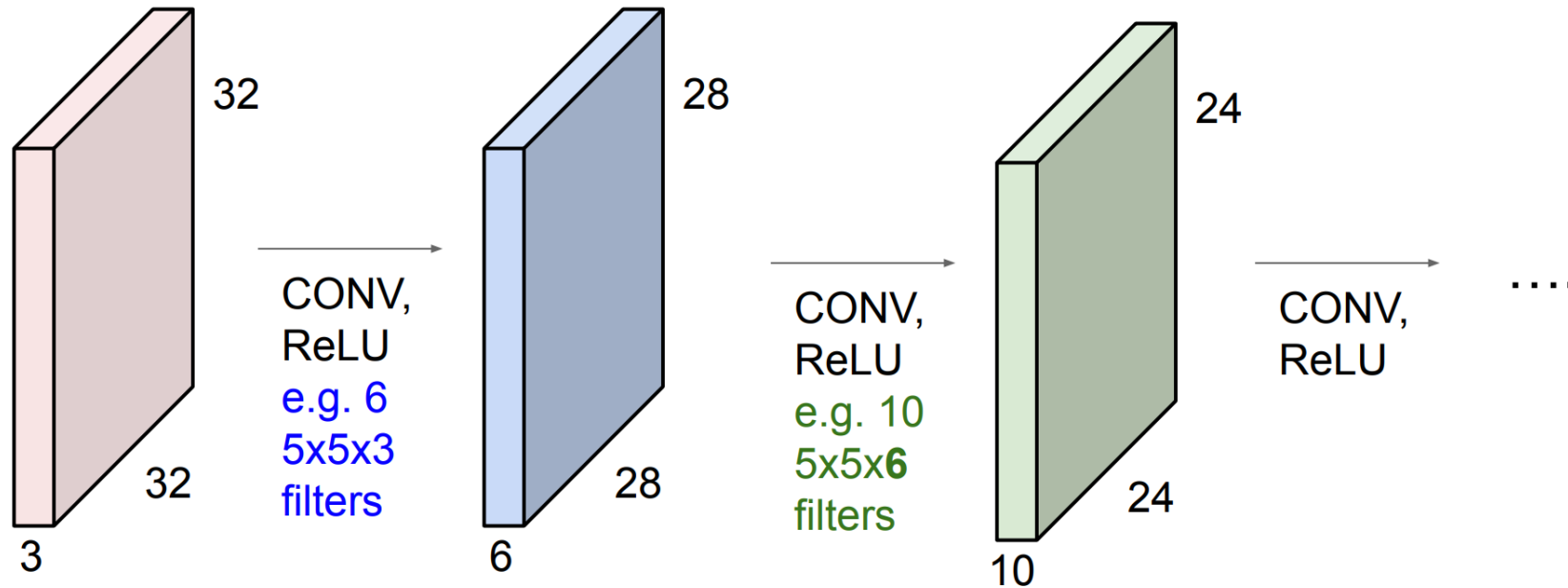


0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	1.77



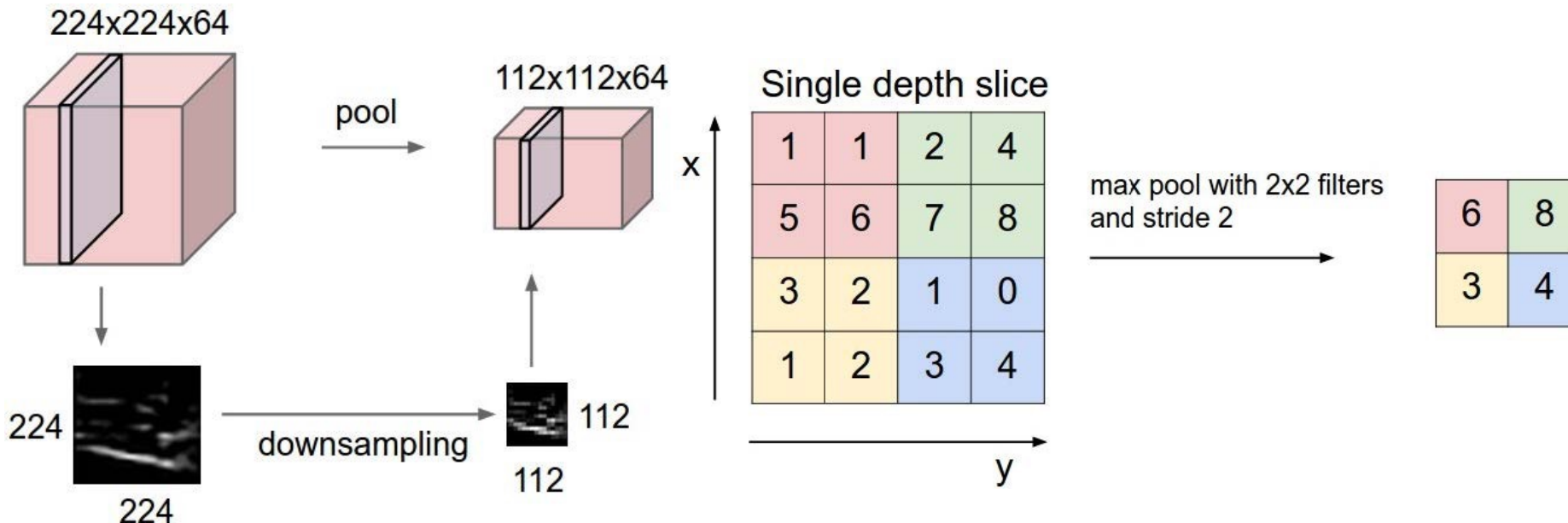
ConvNet

- ConvNet is a sequence of Convolution Layers, interspersed with activation functions



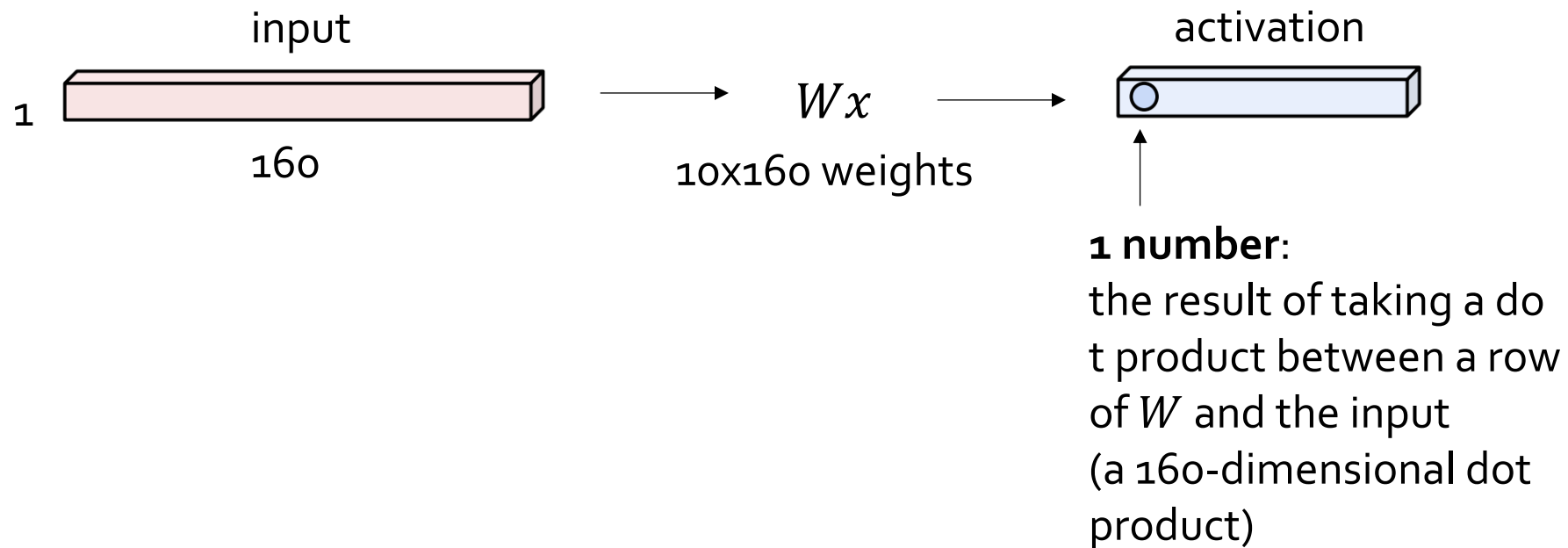
Pooling Layer

- Make the representation smaller and more manageable without losing too much information
 - Operates over each activation map independently
 - Computes MAX operation (most common)
 - Average pooling or L2-norm pooling (less used)



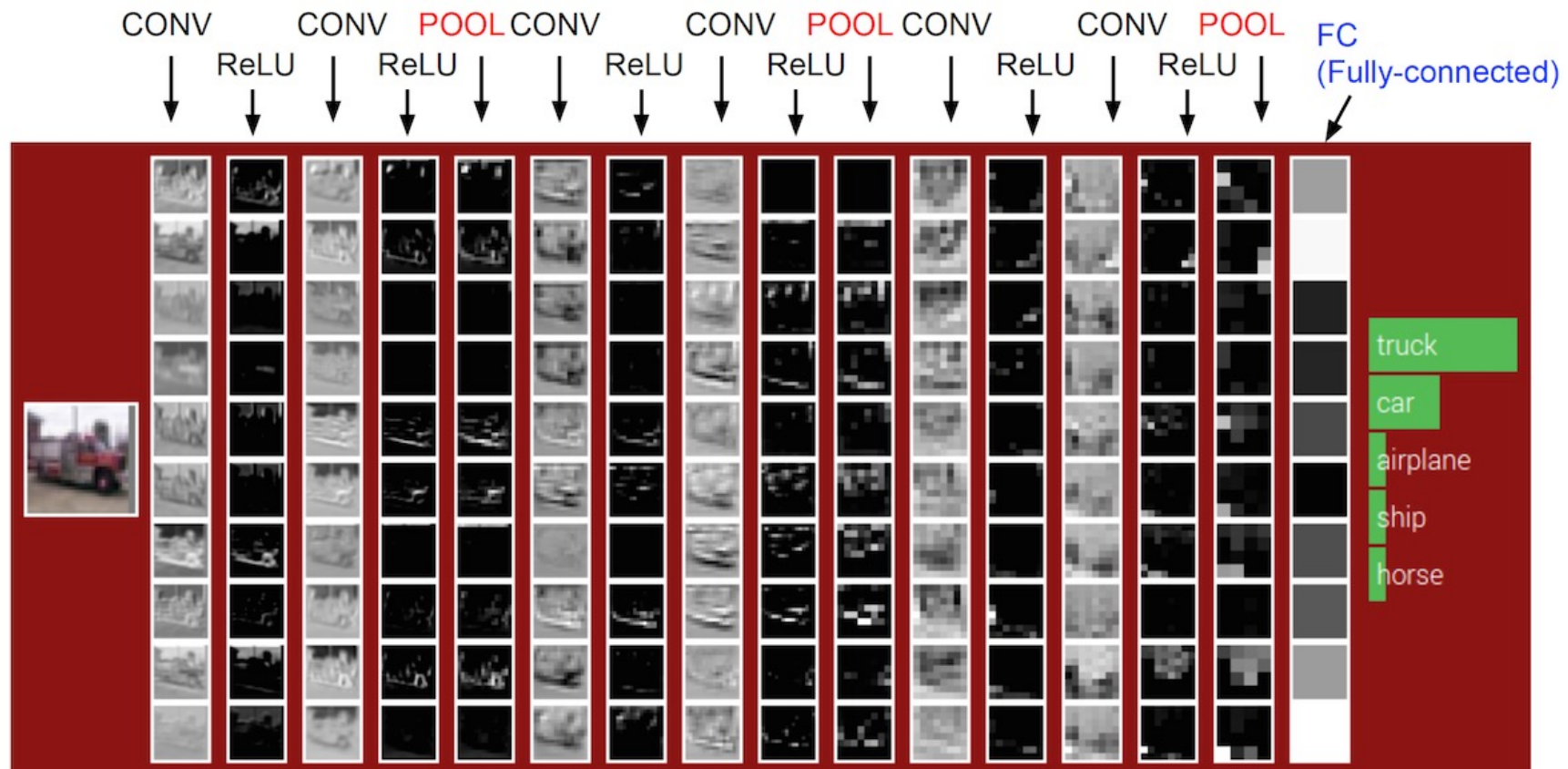
Fully Connected Layer

- Takes the feature map (the output of Conv/POOL layers) as input and predicts the label to describe the image
 - e.g., Given a $4 \times 4 \times 10$ feature map as input, stretch to 160×1



Layer Patterns

- Most common form of ConvNet architecture
 - Stacks a few CONV-RELU layers, follows them with POOL layers, repeats this pattern, finally adds FC layer



CNNs In Practice

- Prefer a stack of small receptive field CONV to one large CONV layer
 - e.g., Three CONV layer consisting of 3x3 filters vs one CONV layer consisting 7x7 filters
 - Which is better? The former is BETTER!
 - 1. Filtering is a linear function over the input. Thus, three stacks of CONV layer contain non-linearity to make features more expressive
 - 2. Fast computation!
 - e.g., If all volumes have depths of C ,
 - 7×7 CONV layer: $C \times (7 \times 7 \times C) = 49C^2$
 - 3 3×3 CONV layer: $3 \times (C \times (3 \times 3 \times C)) = 27C^2$

Rule of Thumbs for Sizing CNNs

- Input layer: divisible by 2 many times
 - 32 for CIFAR-10, 64 or 96 for STL-10, 224, 384, 512 for AlexNet
- CONV layer: use small filters (3x3 or at most 5x5) with zero-padding of $P=1$ at stride of $S=1$
 - Smaller strides work better in practice
 - ex. Input volume of size $[W_i \times H_i \times D_i]$ using K number of $F \times F$ filters at stride of S
 - Output volume: $[W_o \times H_o \times D_o]$, where $W_o = (W_i - F)/S + 1$, $H_o = (H_i - F)/S + 1$, $D_o = K$
- POOL layer: use pooling size of 2x2 with stride of $S=2$
 - 3/4 of the activations will be discarded
- Go deep! use small ones multiple times instead of big

Sizing Example

- Input layer: $[32 \times 32 \times 3]$
- 1. CONV with 10 3×3 filters, stride 1, pad 1 $\rightarrow [32 \times 32 \times 10]$
 - New parameters: $(3 \times 3 \times 3) \times 10 + 10 = 280$
- 2. ReLU
- 3. CONV with 10 3×3 filters, stride 1, pad 1 $\rightarrow [32 \times 32 \times 10]$
 - New parameters: $(3 \times 3 \times 10) \times 10 + 10 = 910$
- 4. ReLU
- 5. POOL with 2×2 filters, stride 2 $\rightarrow [16 \times 16 \times 10]$
 - New parameters: 0



Sizing Example

- 6. CONV with 10 3x3 filters, stride 1, pad 1 → [16x16x10]
 - New parameters: $(3*3*10)*10+10 = 910$
- 7. ReLU
- 8. CONV with 10 3x3 filters, stride 1, pad 1 → [16x16x10]
 - New parameters: $(3*3*10)*10+10 = 910$
- 9. ReLU
- 10. POOL with 2x2 filters, stride 2 → [8x8x10]
 - New parameters: 0



Sizing Example

- 11~15. Once again CONV-RELU-CONV-RELU-POOL $\rightarrow [4 \times 4 \times 10]$
- 16. FC layer to 10 neurons (each of which for class score)
 - New parameters: $10 \times 4 \times 4 \times 10 + 10 = 1610$
- Done!
- GPU memory constraints!
 - Ex. $224 \times 224 \times 3$ image with three 3×3 CONV layers with 64 filters each and padding 1
 $\rightarrow 3 \times [224 \times 224 \times 64] = 72\text{MB}$
 - Most GPUs have 3/4/6GB memory
Tesla K40 = 12GB!

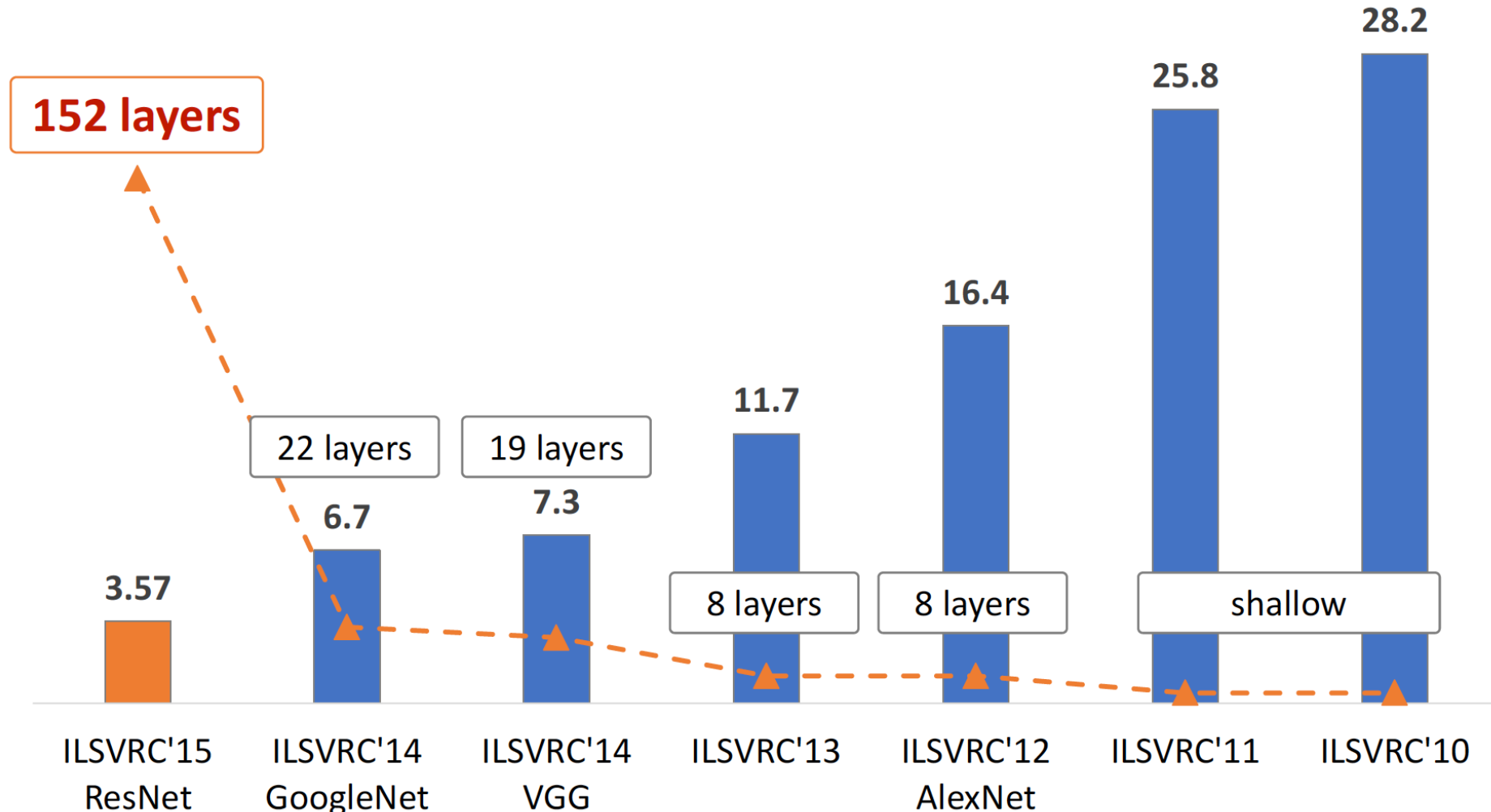


Outline

- Filters and Convolution
- Convolutional Neural network
- **Case Studies**
 - Old Models
 - VGGNet, GoogleNet
 - ResNet

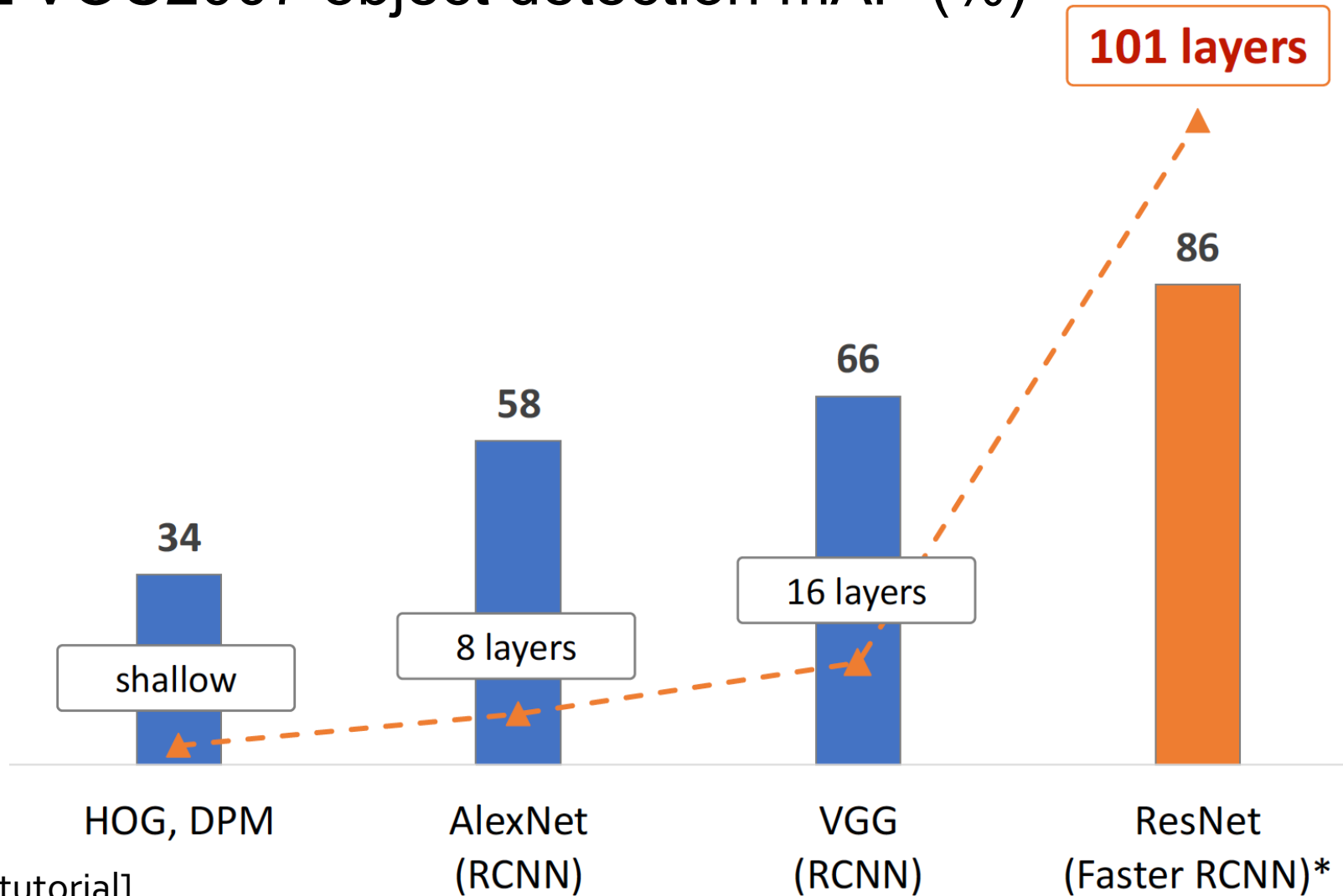
Revolution of Depth

- ImageNet classification top-5 error (%)



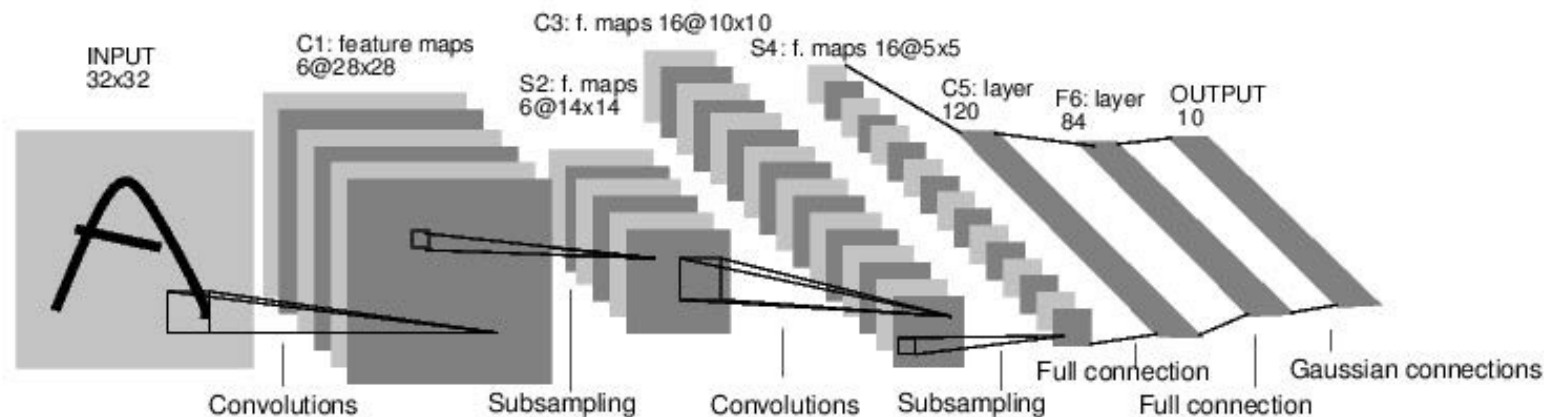
Revolution of Depth

- PASCAL VOC2007 object detection mAP (%)



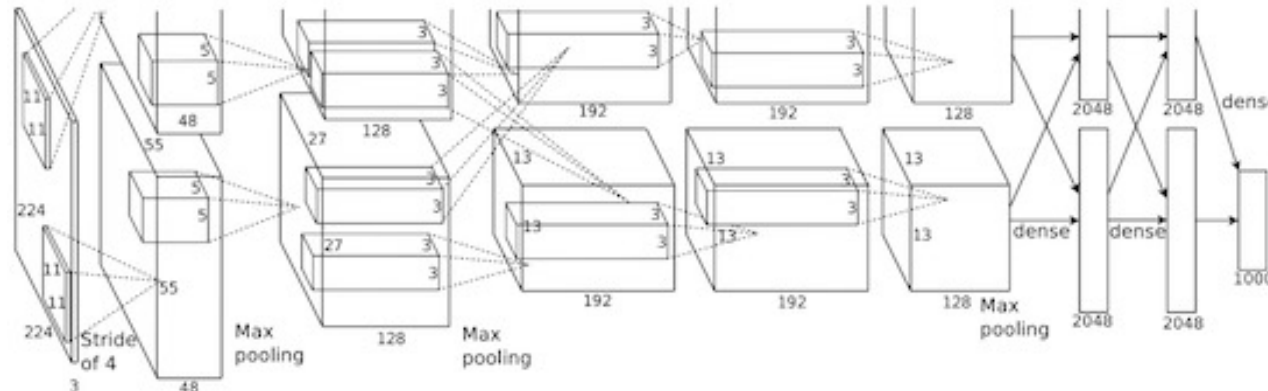
LeNet

- First successful CNN for recognition of hand-written digits
- Propose basic components of modern ConvNets
 - Convolution: locally-connected, spatially weight-sharing (5x5 at stride 1)
 - Subsampling (pooling) (2x2 at stride 2)
 - [CONV-POOL-CONV-POOL-FC-FC]
 - Training by Backpropagation



AlexNet

- The first work revolutionized CNN in computer Vision
 - Similar to LeNet, but deeper, bigger, and featured layers of CONV, RELU, POOL
 - In ImageNet ILSVRC challenge 2012, significantly outperform the runner-up (top 5 error of $16\% < 26\%$)
 - ReLU: accelerate training and better grad prop (vs. tanh)
 - Dropout: In-network ensembling for reducing overfitting
 - Heavy data augmentation: Label-preserving transformation



AlexNet

Architecture details

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV₁**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL₁**: 3x3 filters at stride 2

[27x27x96] **NORM₁**: Normalization layer

[27x27x256] **CONV₂**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL₂**: 3x3 filters at stride 2

[13x13x256] **NORM₂**: Normalization layer

[13x13x384] **CONV₃**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV₄**: 384 3x3 filters at stride 1, pad 1

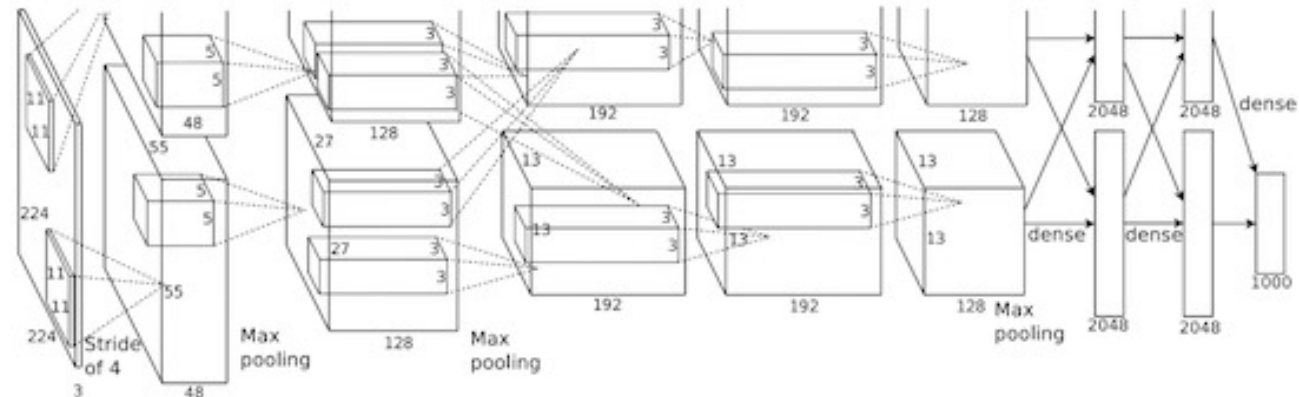
[13x13x256] **CONV₅**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL₃**: 3x3 filters at stride 2

[4096] **FC₆**: 4096 neurons

[4096] **FC₇**: 4096 neurons

[1000] **FC₈**: 1000 neurons
(class scores)

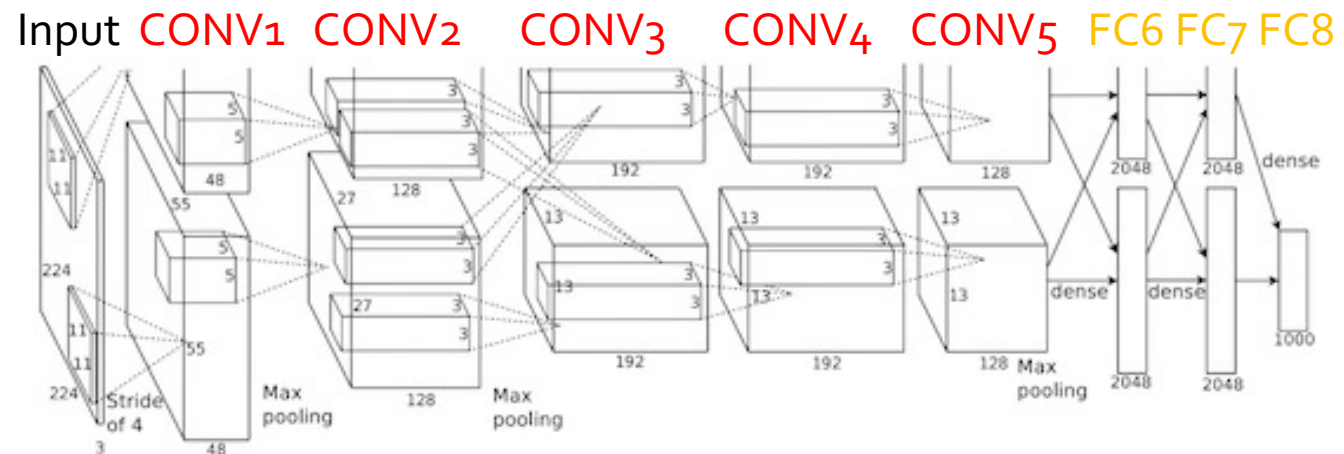


Other details

- Used Norm layers (not common anymore)
- Dropout 0.5
- Batch size 128
- SGD Momentum 0.9
- Learning rate 1e-2, reduced by 10 manually when val acc. plateaus
- L2 weight decay 5e-4
- 7 CNN ensemble: 18.2% -> 15.4%

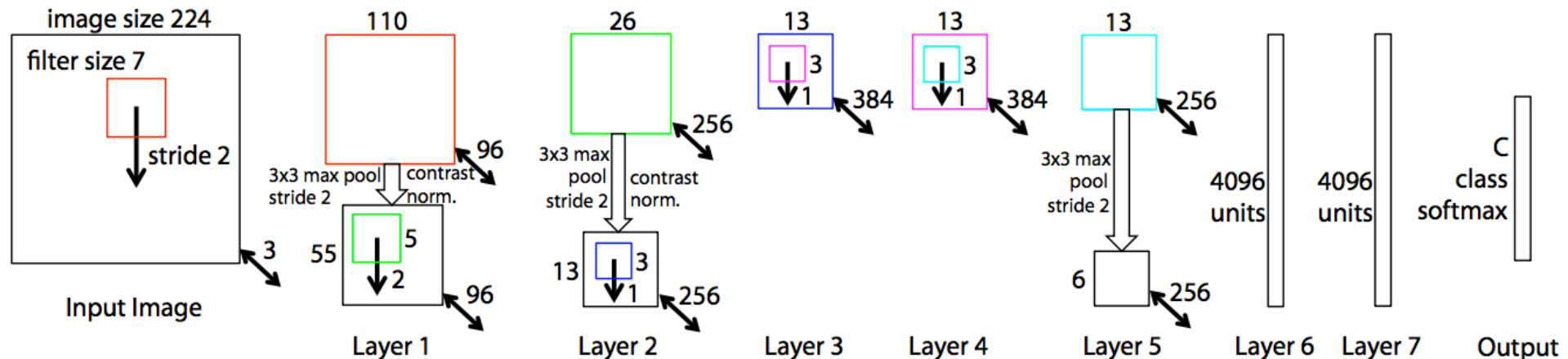
AlexNet

- Implementation details
 - Trained on GTX 580 GPU with only 3 GB of memory
 - Network spread across 2 GPUs, half the neurons (feature maps) on each GPU
 - CONV_{1/2/4/5}: Connections only with feature maps on same GPU
 - CONV₃, FC_{6/7/8}: Connections with all feature maps in preceding layer, communication across GPUs



ZFNet

- Engineered AlexNet
 - The winner of ILSVRC 2013 (ImageNet Top-5: 16.4% → 11.7%)
 - e.g. Compared to AlexNet, CONV₁: (11x11 stride 4) to (7x7 stride 2)
CONV_{3,4,5}: (384, 384, 256) filters to (512, 1024, 512)
 - Not popularly used these days

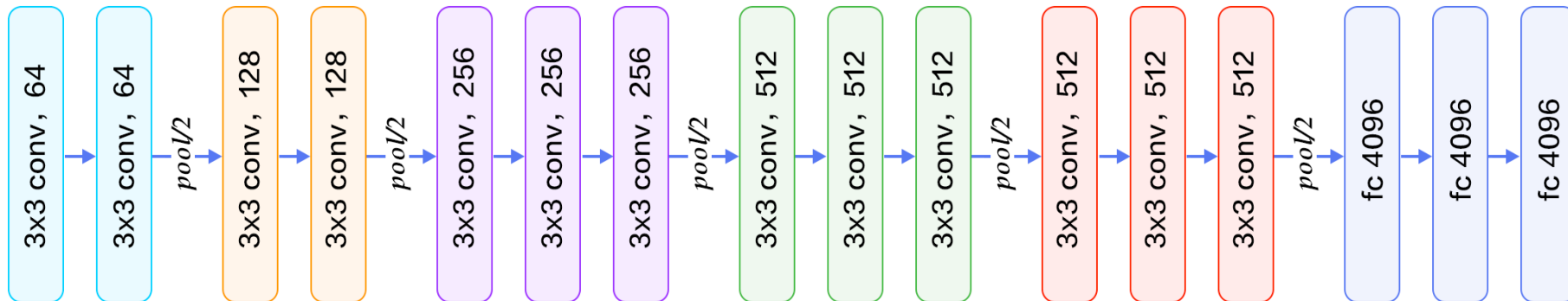


Outline

- Filters and Convolution
- Convolutional Neural network
- Training Tips
- **Case Studies**
 - Old Models
 - VGGNet, GoogleNet
 - ResNet

VGGNet

- Deeper versions of AlexNet (8 layers)
 - The runner-up of ILSVRC 2014 (ImageNet Top-5: 11.7% $\rightarrow$ 7.3%)
 - Good performance but not so efficient
 - Modularized design: Stack the same module (3x3 Conv stride 1, pad 1)
 - Same computation per module (1/2 spatial size $\rightarrow$ 2x filters)
 - Stage-wise training: VGG-11 $\rightarrow$ VGG-13 $\rightarrow$ VGG-16



VGGNet

Most memory is in early CONV

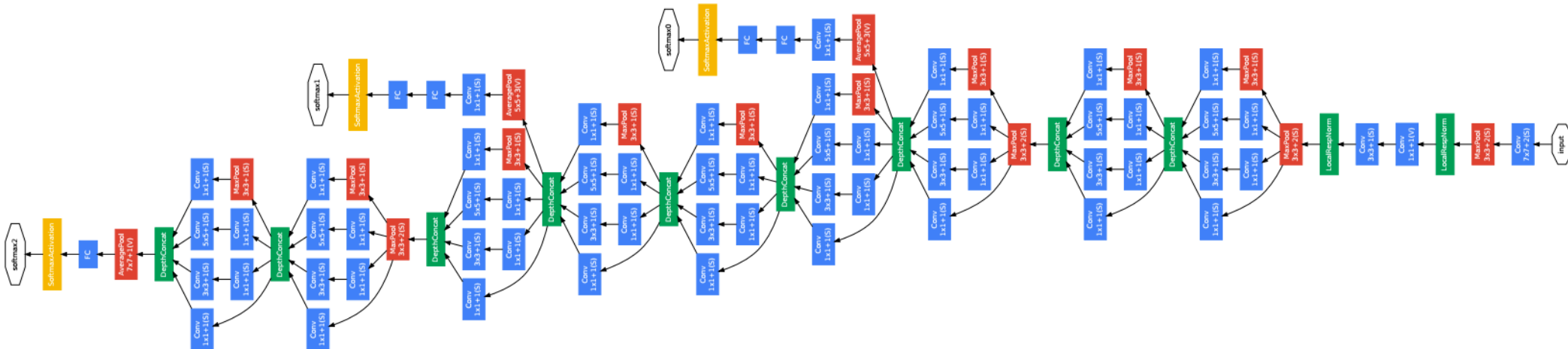
TOTAL memory: $24M * 4 \text{ bytes} \approx 96MB/\text{image}$
TOTAL params: 138M parameters

INPUT: [224x224x3]	memory: $224 * 224 * 3 = 150K$	params: 0
CONV3-64: [224x224x64]	memory: $224 * 224 * 64 = 3.2M$	params: $(3 * 3 * 3) * 64 = 1,728$
CONV3-64: [224x224x64]	memory: $224 * 224 * 64 = 3.2M$	params: $(3 * 3 * 64) * 64 = 36,864$
POOL2: [112x112x64]	memory: $112 * 112 * 64 = 800K$	params: 0
CONV3-128: [112x112x128]	memory: $112 * 112 * 128 = 1.6M$	params: $(3 * 3 * 64) * 128 = 73,728$
CONV3-128: [112x112x128]	memory: $112 * 112 * 128 = 1.6M$	params: $(3 * 3 * 128) * 128 = 147,456$
POOL2: [56x56x128]	memory: $56 * 56 * 128 = 400K$	params: 0
CONV3-256: [56x56x256]	memory: $56 * 56 * 256 = 800K$	params: $(3 * 3 * 128) * 256 = 294,912$
CONV3-256: [56x56x256]	memory: $56 * 56 * 256 = 800K$	params: $(3 * 3 * 256) * 256 = 589,824$
CONV3-256: [56x56x256]	memory: $56 * 56 * 256 = 800K$	params: $(3 * 3 * 256) * 256 = 589,824$
POOL2: [28x28x256]	memory: $28 * 28 * 256 = 200K$	params: 0
CONV3-512: [28x28x512]	memory: $28 * 28 * 512 = 400K$	params: $(3 * 3 * 256) * 512 = 1,179,648$
CONV3-512: [28x28x512]	memory: $28 * 28 * 512 = 400K$	params: $(3 * 3 * 512) * 512 = 2,359,296$
CONV3-512: [28x28x512]	memory: $28 * 28 * 512 = 400K$	params: $(3 * 3 * 512) * 512 = 2,359,296$
POOL2: [14x14x512]	memory: $14 * 14 * 512 = 100K$	params: 0
CONV3-512: [14x14x512]	memory: $14 * 14 * 512 = 100K$	params: $(3 * 3 * 512) * 512 = 2,359,296$
CONV3-512: [14x14x512]	memory: $14 * 14 * 512 = 100K$	params: $(3 * 3 * 512) * 512 = 2,359,296$
CONV3-512: [14x14x512]	memory: $14 * 14 * 512 = 100K$	params: $(3 * 3 * 512) * 512 = 2,359,296$
POOL2: [7x7x512]	memory: $7 * 7 * 512 = 25K$	params: 0
FC: [1x1x4096]	memory: 4096	params: $7 * 7 * 512 * 4096 = 102,760,448$
FC: [1x1x4096]	memory: 4096	params: $4096 * 4096 = 16,777,216$
FC: [1x1x1000]	memory: 1000	params: $4096 * 1000 = 4,096,000$

Most params are in late FC

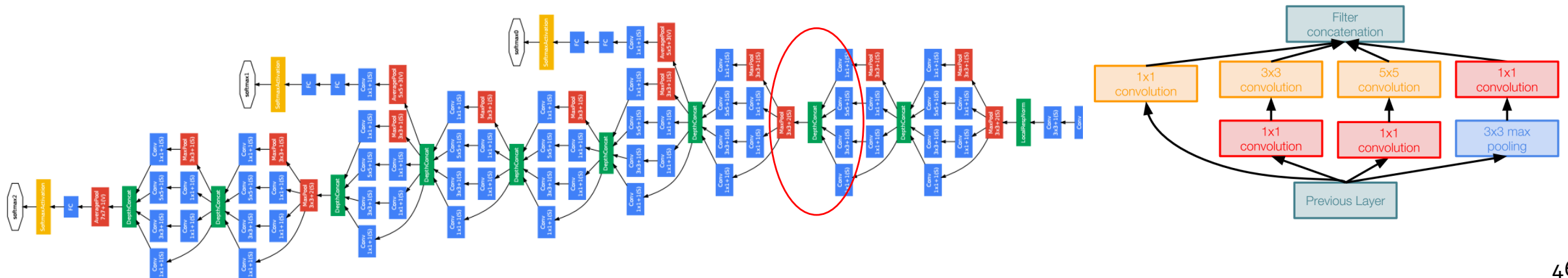
GoogLeNet

- Inception (network in network) structure
 - The winner of ILSVRC 2014 (ImageNet Top-5: 7.3% → 6.7%)
 - Much fewer parameters of 5M compared to 60M of AlexNet
 - No FC layers and 22 layers in total (v1)
 - Keep updating... to Inception-v4 (Inception-ResNet)



GoogLeNet

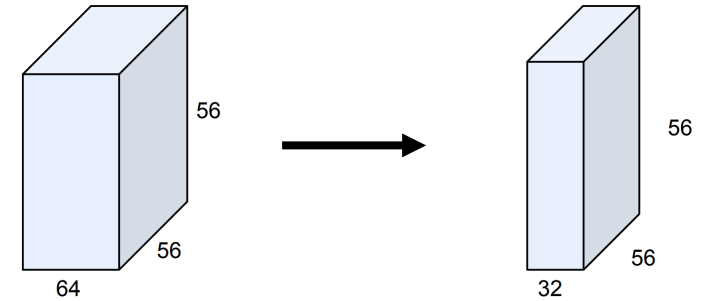
- Key ideas of Inception module
 - Design a good local network topology (network within a network) and then stack these modules on top of each other
 - Multiple branches: e.g., 1×1 , 3×3 , 5×5 , pool
 - Shortcuts: stand-alone 1×1
 - Bottleneck: reduce dim by 1×1 before expensive $3\times 3/5\times 5$ conv
 - Concatenate all filter outputs together depth-wise



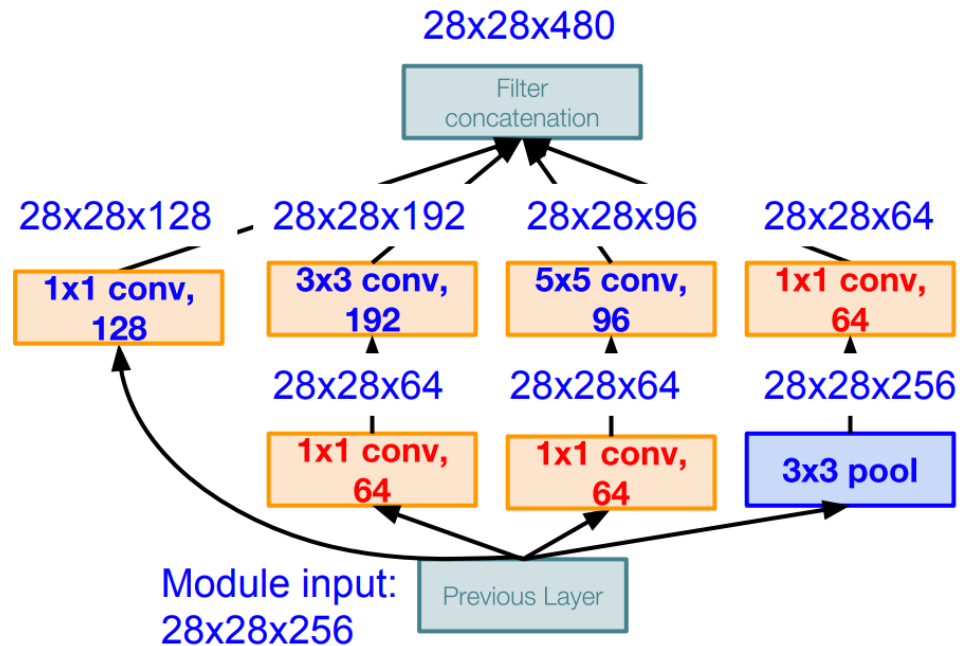
GoogLeNet

- Inception module with dimension reduction
 - Compared to 854M ops for naive version, Bottleneck can reduce depth after pooling layer

cf) 1x1 CONV with 32 filters



Each filter has size 1x1x64, and performs a 64-dim dot product

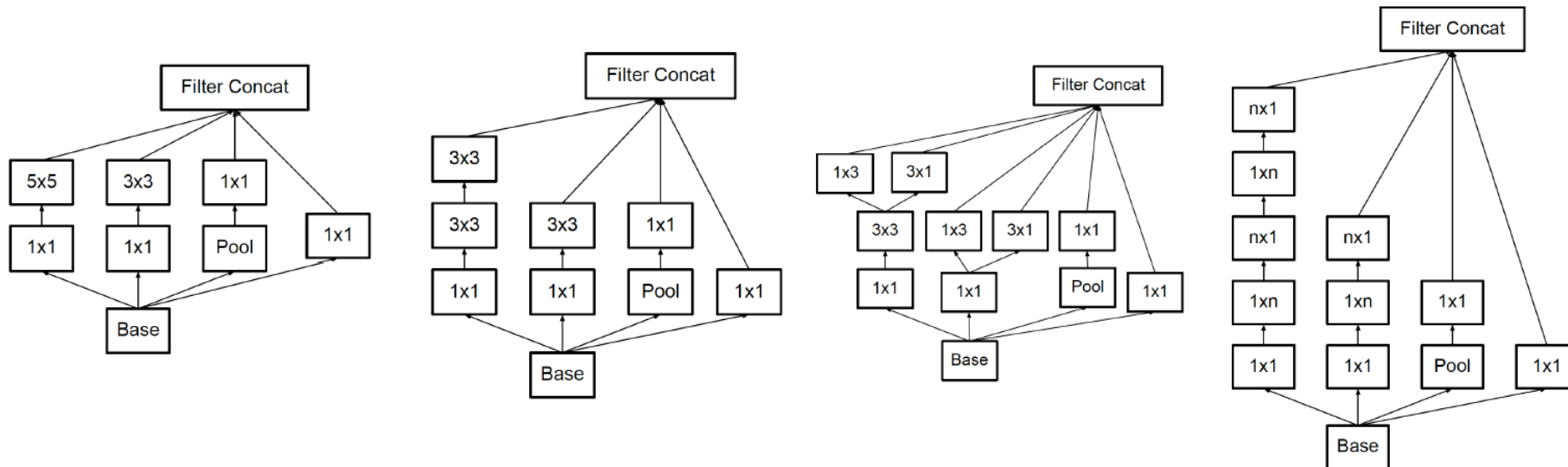


Conv Ops:

[1x1 conv, 64]	28x28x64x1x1x256
[1x1 conv, 64]	28x28x64x1x1x256
[1x1 conv, 128]	28x28x128x1x1x256
[3x3 conv, 192]	28x28x192x3x3x64
[5x5 conv, 96]	28x28x96x5x5x64
[1x1 conv, 64]	28x28x64x1x1x256
Total: 358M ops	

GoogLeNet / Inception v1-v3

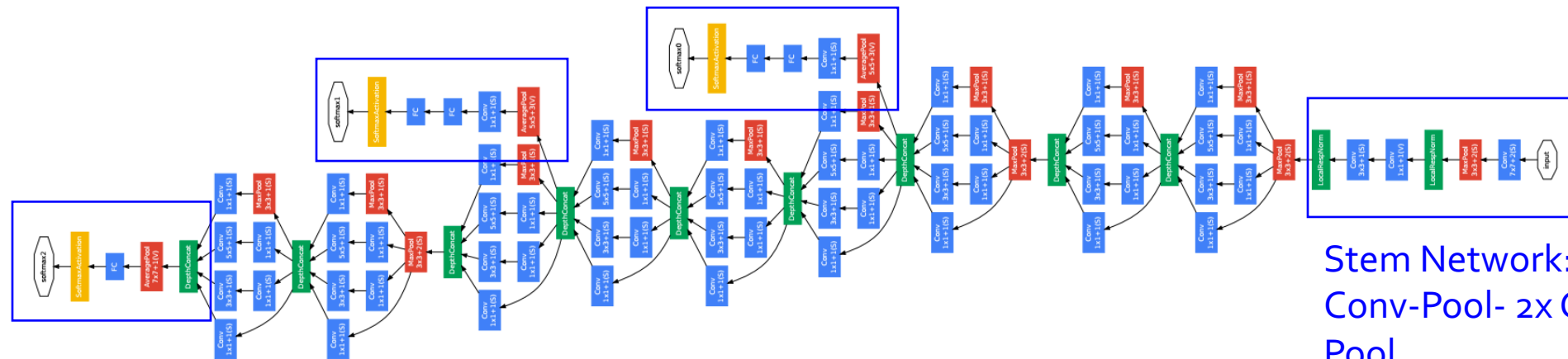
- More templates, but the same 3 properties are kept
 - Multiple branches, shortcuts, bottleneck
- Optimizing multi-branch CNNs largely benefits from BN
 - Including all Inceptions and ResNets



GoogLeNet

- Full GoogLeNet architecture
 - Stack Inception modules with dim reduction on top of each other
 - 22 total layers with weights (parallel layers count as 1 → 2 layers per Inception module. Don't count auxiliary output layers)

Auxiliary classification outputs to inject additional gradient at lower layers (AvgPool-1x1Conv-FC-FC-Softmax)



Classifier output (removed expensive FC layers!)

Stacked inception modules

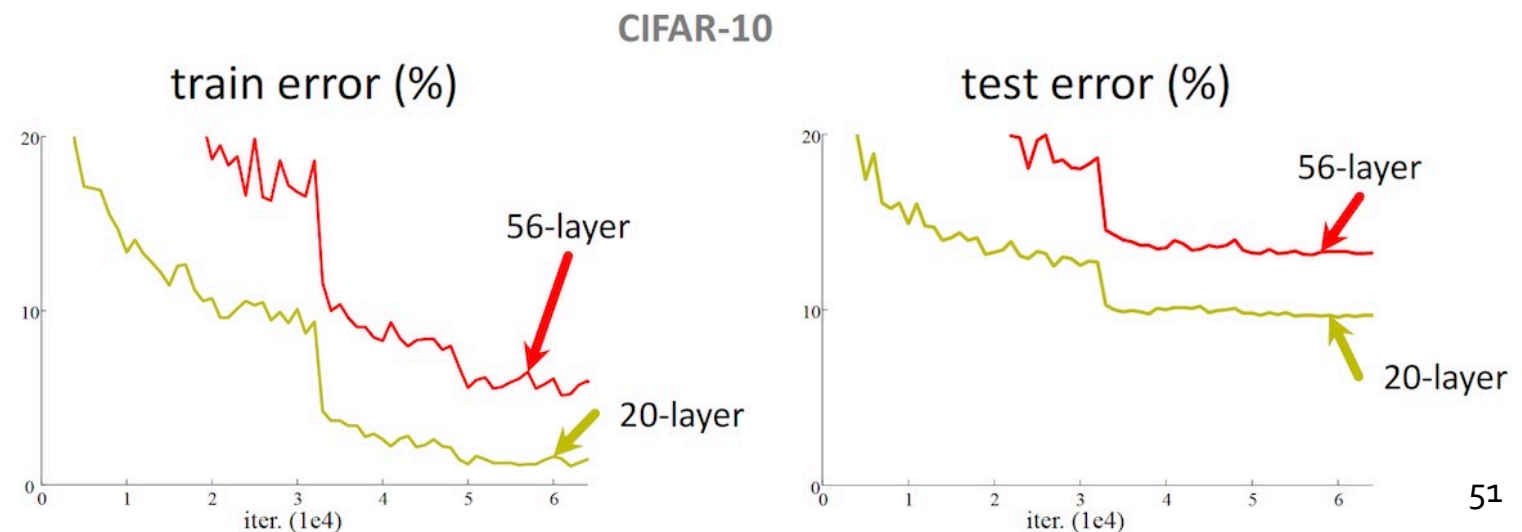
Stem Network:
Conv-Pool- 2x Conv-
Pool

Outline

- Filters and Convolution
- Convolutional Neural network
- Training Tips
- **Case Studies**
 - Old Models
 - VGGNet, GoogleNet
 - **ResNet**

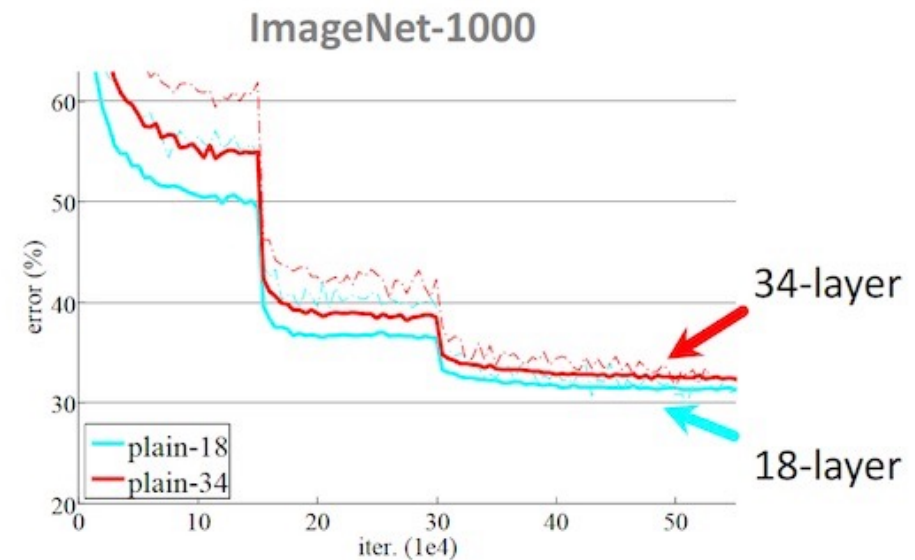
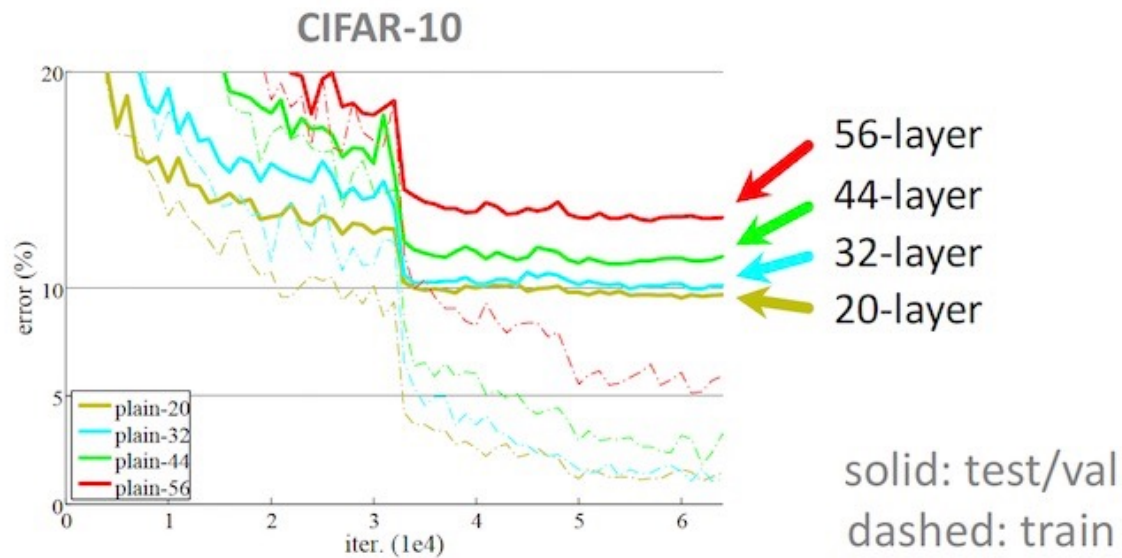
ResNet – Degradation Problem

- Learn better networks by simply stacking more layers?
- Plain nets: stacking 3x3 conv layers...
 - With the network depth increasing, accuracy gets saturated and then degraded rapidly
- NOT caused by **overfitting**
 - 56-layer net has higher **training and test error** than 20-layer net

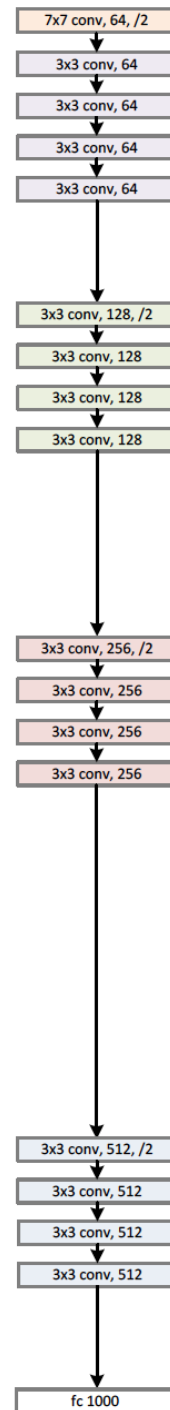


ResNet – Degradation Problem

- Overly deep plain nets have **higher training** error
- A general phenomenon, observed in many datasets

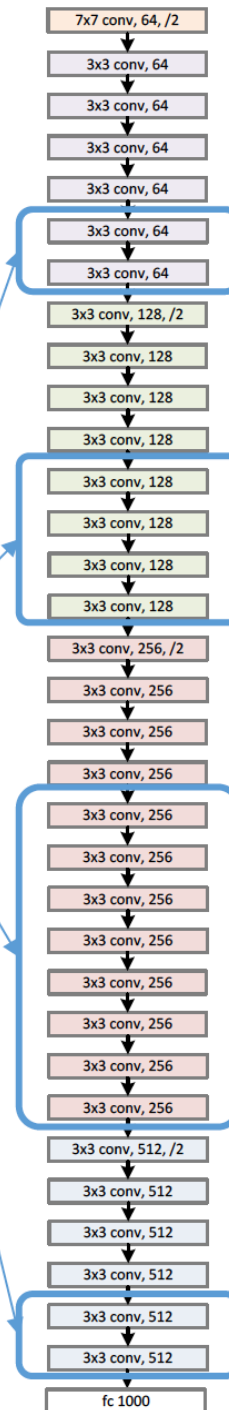


A shallower model
(18 layers)



A deeper counterpart
(34 layers)

“extra”
layers



A richer solution space

A deeper model should not
have higher **training error**

A solution by construction

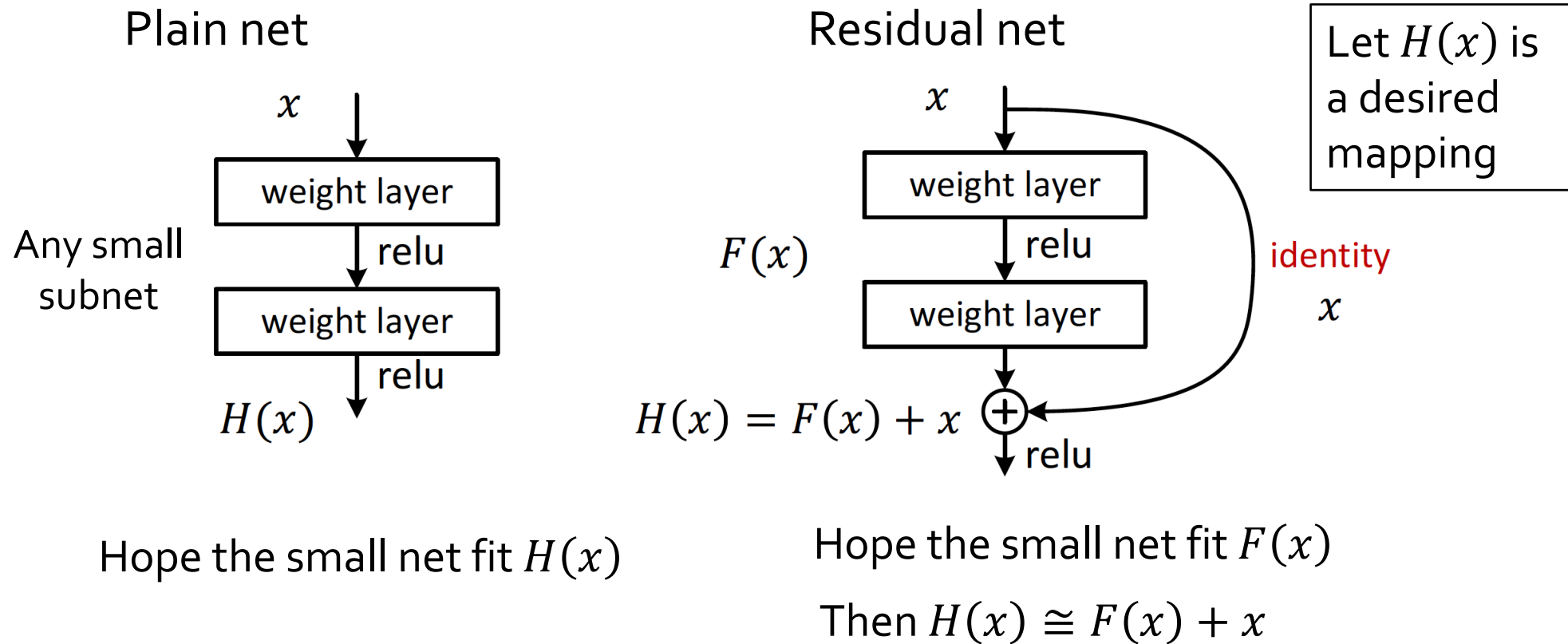
- Original layers: copied from a learned shallower model
- Extra layers: set as **identity**
- At least same training error

(Hypothesis)

Optimization difficulties: solvers cannot find the solution when going deeper...

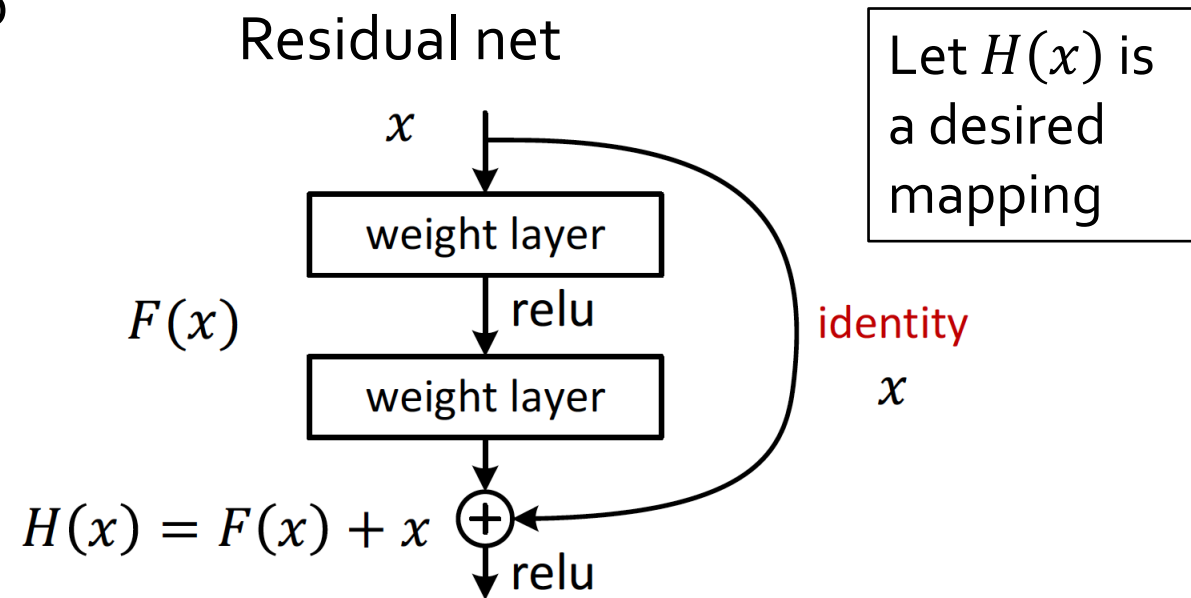
Deep Residual Learning

- Feedforward neural networks with shortcut connections (i.e., identity mapping)



Deep Residual Learning

- $F(x)$ is a residual mapping w.r.t. **identity**
 - Focus on representing additional info from what previous layers already learned
 - If identity were optimal, easy to set weights as 0 (cf. Identity mappings that plain models have difficulty in representing)

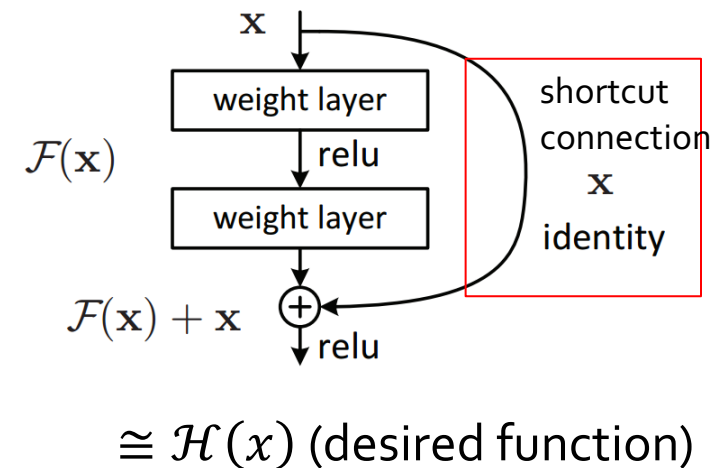


Hope the small net fit $F(x)$

Then $H(x) \cong F(x) + x$

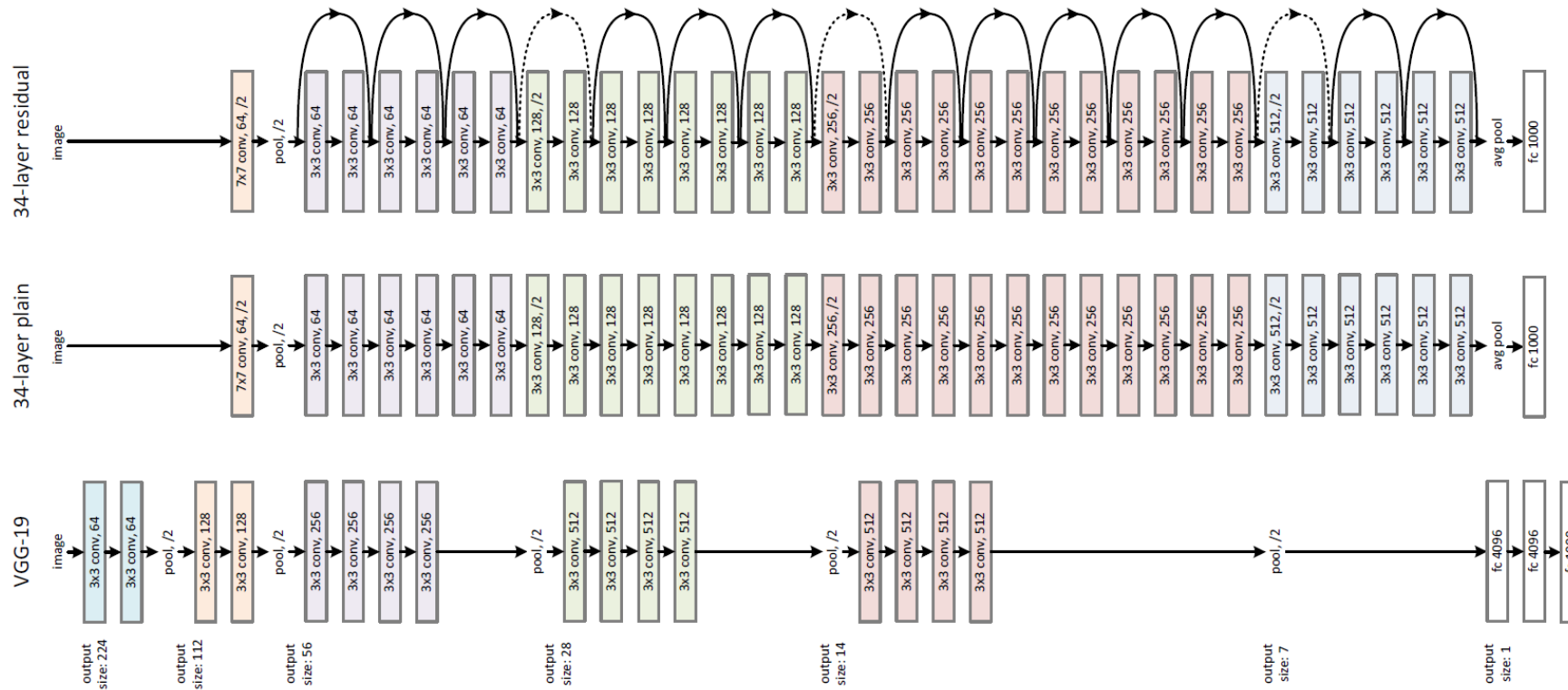
Advantage of Residual Learning

- Identity shortcut connections add neither extra parameter nor computational complexity
- Easy end-to-end optimization by backpropagation
- Accuracy gains from greatly increased depth
 - 152-layer ResNet achieves 3.57% top-5 errors on ImageNet datasets with lower complexity than VGGNet



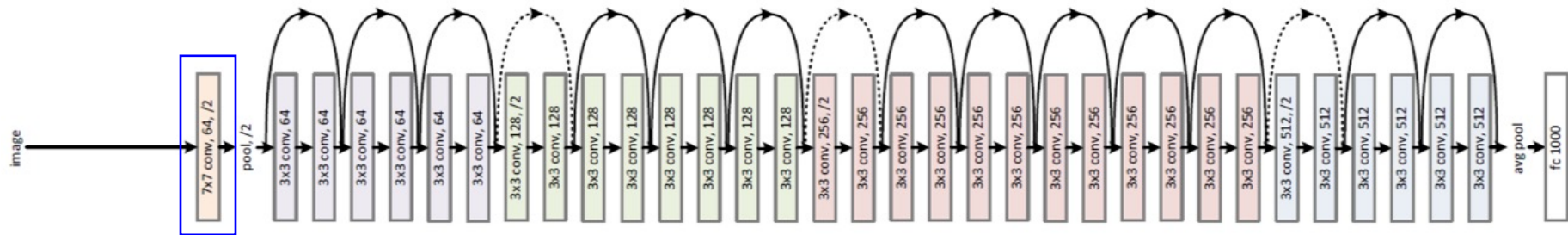
34-Layer ResNet

- Design principles – similar to VGG-19
 - Same feature map size, same # filters (channels)
 - Whenever feature map size is halved, # filters is doubled to preserve the time complexity per layer



34-Layer ResNet

- Full ResNet architecture (stack residual blocks)
 - Every residual block has two 3x3 conv layers
 - Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
 - 34-layer ResNet has only 19% FLOPs of VGG-19: (i) 1/2 input size (224/14 -> 112/7), (ii) No hidden FC layer
 - Total depths of 34, 50, 101, or 152 layers for ImageNet



Additional conv layer
at the beginning

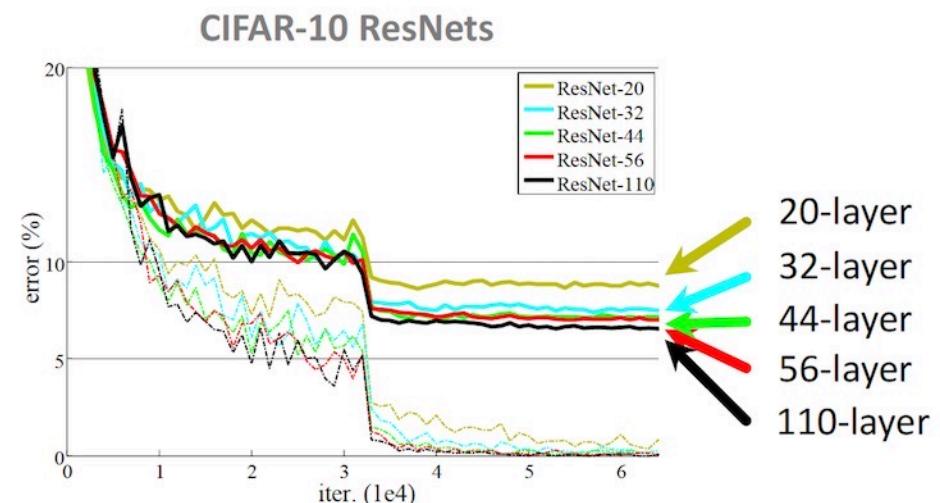
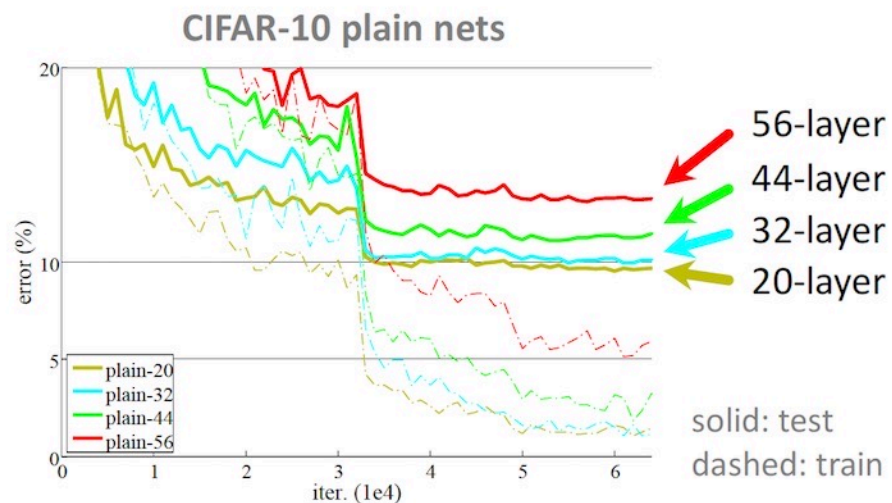
No FC layers except
fc1000 to output classes

34-Layer ResNet

- Comparing 18/34-layer residual/plain on ImageNet
 - Plain-18/34: degradation problem
 - ResNet-18/34: ResNet-34 better
 - Plain/ResNet-18: ResNet-18 converges faster
 - Deeper ResNets have **lower training error**, and also lower test error

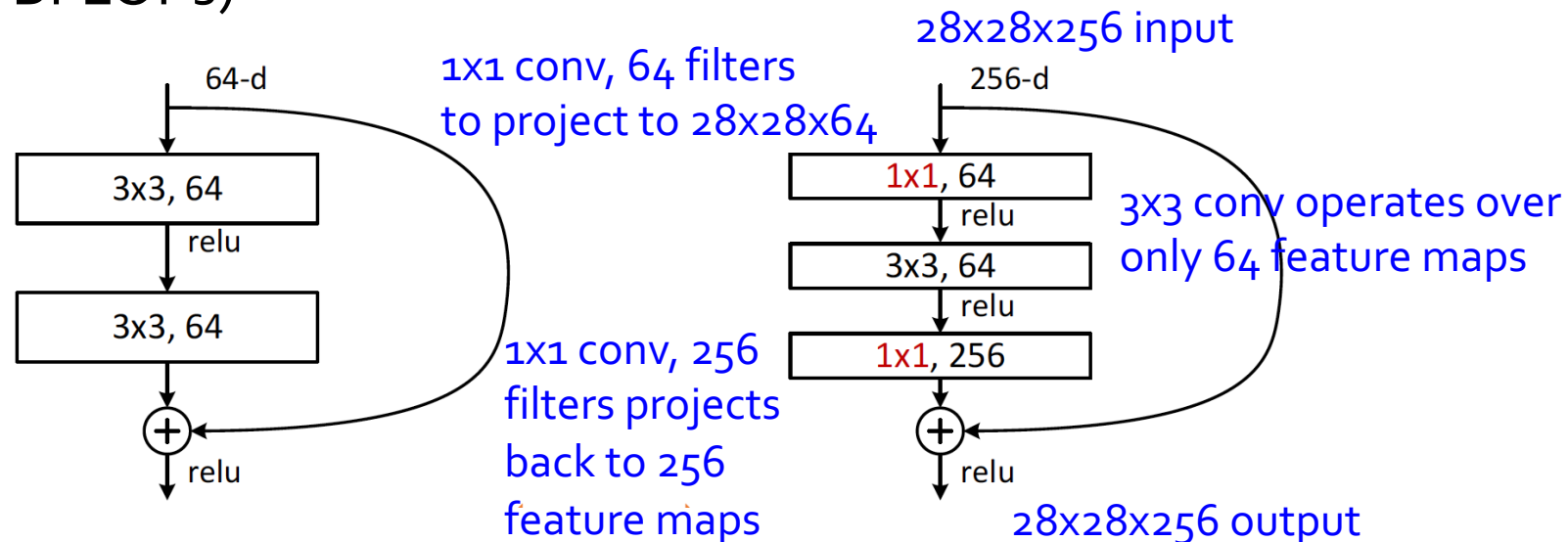
Top-1 error on ImageNet validation

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03



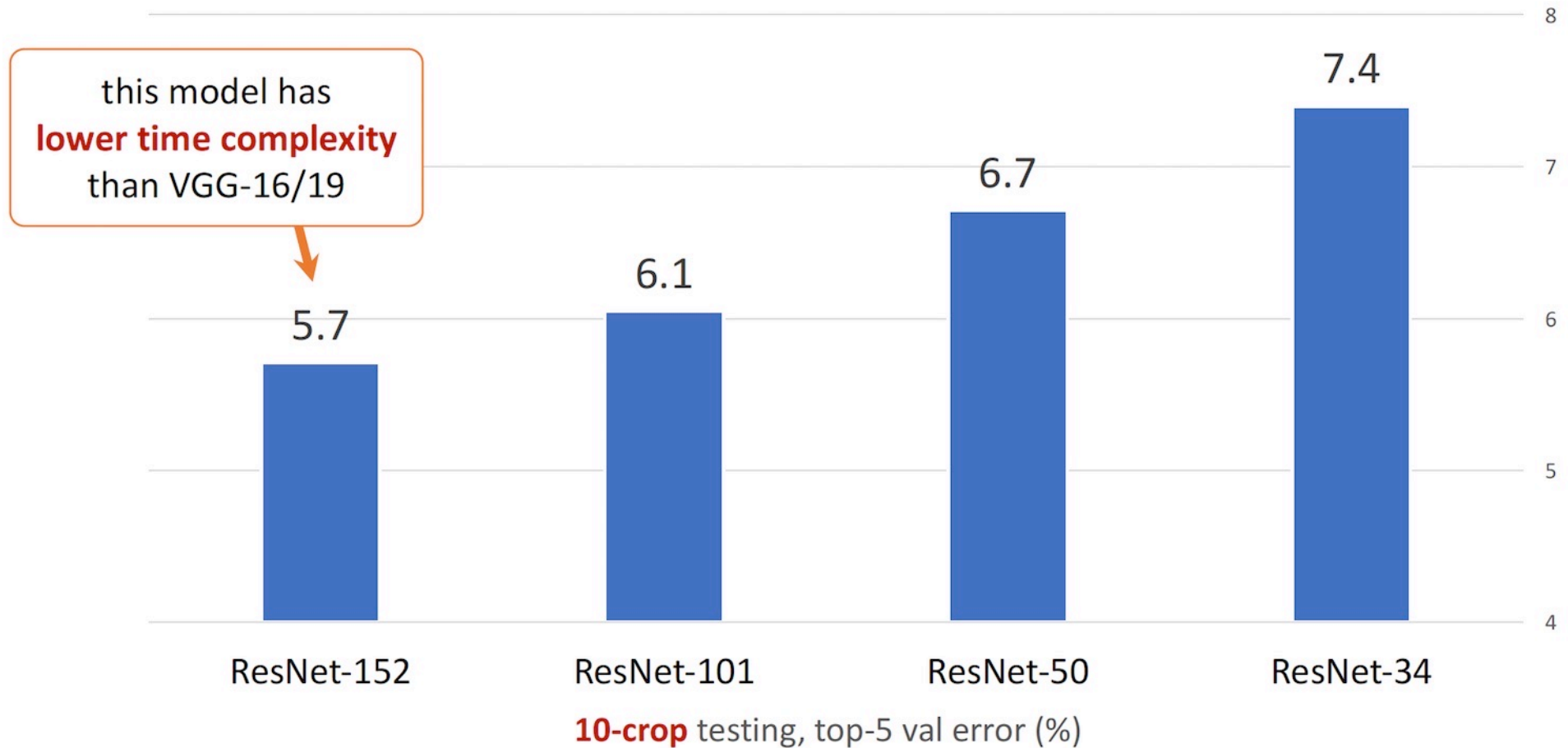
A Practical Design of Deeper ResNet

- Deeper bottleneck architectures (ResNet-50+)
 - Two 3x3 CONV layers $\rightarrow$ 1x1, 3x3, 1x1 layers
 - Use bottleneck 1x1 layers – reducing and increasing dimensions
 - Both designs have similar time complexity
 - For 50/101/152-layers: 152-layer ResNet still has lower complexity than VGG-19 (15.3/19.6 BFLOPs)



ImageNet Experiments

- Deeper ResNet has **lower** errors



ResNet Summary

- Swept 1st place in all ILSVRC&COCO 2015 competitions
 - ImageNet classification/detection/localization, COCO detection/segmentation
 - Able to train very deep networks without degrading (152 layers on ImageNet, 1202 on CIFAR)
 - ImageNet Top-5: 6.7% → 3.5% better than human (5.1%)

MSRA @ ILSVRC & COCO 2015 Competitions

- **1st places in all five main tracks**

- ImageNet Classification: “Ultra-deep” (quote Yann) **152-layer** nets
- ImageNet Detection: **16%** better than 2nd
- ImageNet Localization: **27%** better than 2nd
- COCO Detection: **11%** better than 2nd
- COCO Segmentation: **12%** better than 2nd

Comparing Complexity

Inception-v4: ResNet + Inception

- VGG: Highest memory, most operations
- GoogLeNet: most efficient
- AlexNet: Smaller compute, still memory heavy, lower accuracy

